LOG ANALYTICS SYSTEM FOR A WEB SERVER USING HADOOP HDFS AND APACHE SPARK

A PROJECT REPORT

(Complete Project Report)

Submitted by

KUSHAL MOHAN [RA2411028010082]
KRITI SINGH [RA2411028010081]
GARGI SINGH [RA2411028010102]

Under the Guidance of

Dr. SENTHIL KUMAR K
Designation, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Cloud Computing



DEPARTMENT OF NETWORKING AND COMMUNICATIONS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSC314P – Big Data Essentials mini-project report titled **"LOG ANALYTICS SYSTEM FOR A WEB SERVER USING HADOOP HDFS AND APACHE SPARK"** is the bonafide work of "KUSHAL MOHAN [RA2411028010082], KRITI SINGH [RA2411028010081], GARGI SINGH [RA2411028010102]" who carried out the mini-project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Dr. SENTHIL KUMAR K Designation Department of Networking and Communications SRM Institute of Science and Technology	_____ Head of the Department Department of Networking and Communications SRM Institute of Science and Technology
Panel Reviewer I	**Panel Reviewer II**
SIGNATURE	**SIGNATURE**

Department of Networking and Communications
SRM Institute of Science & Technology
Own Work* Declaration Form

Degree / Course	**: B.Tech CSE-CC / Big Data Essentials**
Student Names	**: Kushal Mohan, Kriti Singh, Gargi Singh**
Registration Numbers	**: RA2411028010082, RA2411028010081, RA2411028010102**
Title of Work	**: LOG ANALYTICS SYSTEM FOR A WEB SERVER USING HADOOP HDFS AND APACHE SPARK**

I / We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism**, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Complied with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Kushal Mohan	Kriti Singh	Gargi Singh
(RA2411028010082)	(RA2411028010081)	(RA2411028010102)

Date: ____ / ____ / 2026

ACKNOWLEDGEMENT

We would like to sincerely thank our faculty guide, Dr. Senthil Kumar K, Department of Networking and Communications at SRM Institute of Science and Technology for the guidance, consistent support and constructive feedback given throughout the development of our project, “Log Analytics System for a Web Server using Hadoop HDFS and Apache Spark.” The guidance we received on distributed storage, data processing and machine learning gave us a strong foundation for approaching this topic with confidence.

We also thank the faculty members of the School of Computing, whose dedication to a collaborative and practical learning environment greatly enriched our experience, and our classmates, whose discussions and shared resources helped us at many points of the project.

We are grateful to the open-source communities behind Apache Hadoop, Apache Spark, Apache Parquet, Apache Superset and Docker, whose software and documentation made this project possible, to the Internet Traffic Archive and the original contributors of the NASA-HTTP traces for making real web server logs freely available, and to the authors of the HttpParams and Fwaf datasets for publishing labelled attack data. Online communities such as Stack Overflow helped us solve many practical problems along the way.

Finally, we thank our families for their constant support and encouragement throughout this academic journey. Through this project we have learned how large, raw data can be stored safely, processed at scale and turned into answers that people can use.

Authors

ABSTRACT

Web servers record every request they receive in access logs, which quickly grow to millions of lines. These logs hold valuable information — traffic peaks, popular pages, broken links, automated clients and attack attempts — but their size and lack of structure make them impractical to analyse with spreadsheets or single-machine scripts. This project presents a Log Analytics System that stores, processes, mines and presents web server access logs with a distributed big-data stack. The system runs as a cluster of ten Docker containers with Hadoop Distributed File System (HDFS) 3.3.6 — one NameNode and two DataNodes with a replication factor of two — Apache Spark 3.5.3, Spark SQL, Spark MLlib, the Spark Thrift Server and Apache Superset 4.1.1. It is demonstrated on the public NASA-HTTP dataset of 3,461,613 requests made to NASA's Kennedy Space Center web server in July and August 1995. The raw logs are stored unchanged in an HDFS raw zone; a Spark ETL job parses every line into typed columns, keeps malformed lines as rejects and writes Snappy-compressed Parquet partitioned by date (3,461,612 rows, 8.6 times smaller than the text); and eleven Spark SQL reports summarise traffic, content, errors and performance. A contextual Z-score in Spark SQL flags 12 unusual hours, 11 of them on 13 July 1995, recovering the STS-70 Space Shuttle launch ($z = 5.90$) from the traffic alone. A Gaussian Mixture Model in Spark MLlib, applied to nine behavioural features of 38,104 hosts, separates 1,829 automated hosts that produce 6.36% of the traffic, without labels. For attack detection, 1.31 million labelled requests from the HttpParams and Fwaf datasets are cleaned and freed of a measured dataset shortcut, and a Factorization Machine on 12,292 character n-gram features reaches an F1 score of 0.9669 (recall 98.5%, 399 false alarms on 261,076 test requests). Compared with seven other MLlib classifiers, it is within 0.002 F1 of the linear SVM inside one dataset and transfers clearly best to a dataset it was not trained on (F1 0.968 against 0.846). All results are registered as 27 tables in the Spark Thrift Server and presented in six Apache Superset dashboards with 56 charts, embedded in the LogLens application, which also runs the saved model on any request typed by a user. The system is reproducible with a few commands and covered by 35 automated tests.

TABLE OF CONTENTS

CHAPTER NO.	TOPIC	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1	**INTRODUCTION**	**1**
	1.1 Problem Statement	1
	1.2 Motivation	1
	1.3 Objectives	2
	1.4 Ideas	2
	1.5 Solution Proposed	3
	1.6 Challenges	3
	1.7 Scope of the Report	4
2	**LITERATURE REVIEW**	**5**
3	**PROPOSED METHODOLOGY**	**8**
	3.1 Data Collection	8
	3.2 Data Preprocessing	9
	3.2.1 Preprocessing of the Web Logs	9
	3.2.2 Preprocessing of the Labelled Attack Data	9
	3.3 Feature Engineering	10
	3.3.1 Columns of the Curated Log Table	10
	3.3.2 Behavioural Features for Bot Detection	10
	3.3.3 Text Features for Attack Detection	11
	3.4 System Architecture	11
	3.5 Analytical Models	12
	3.5.1 Contextual Z-Score Anomaly Detection	12
	3.5.2 Gaussian Mixture Model for Bot Detection	12
	3.5.3 Factorization Machine for Attack Detection	13
	3.5.4 Comparison Classifiers	13
	3.6 Evaluation Metrics	13
	3.7 Visualization Techniques	14
4	**IMPLEMENTATION**	**15**
	4.1 Environment and Cluster Setup	15
	4.2 Data Acquisition and Ingestion into HDFS	16
	4.3 Data Cleaning and Preprocessing using Apache Spark	16
	4.4 Analytical Reports using Spark SQL	18
	4.5 Anomaly Detection with Spark SQL	19

CHAPTER NO.	TOPIC	PAGE NO.
	4.6 Bot Detection with Spark MLlib	19
	4.7 Attack Detection Pipeline with Spark MLlib	20
	4.7.1 Preprocessing Job	20
	4.7.2 Feature Pipeline and Training	21
	4.7.3 Hyper-parameter Tuning	22
	4.7.4 Stability, Transfer and Model Saving	22
	4.8 Serving Layer: Spark Thrift Server and Apache Superset	22
	4.9 LogLens Application	23
	4.10 Verification and Testing	24
	4.11 Division of Work	24
	4.12 System Summary	25
5	**RESULTS AND DISCUSSIONS**	**26**
	5.1 Data Processing Results	26
	5.2 Exploratory Data Analysis Results	26
	5.2.1 Traffic by Hour of Day	26
	5.2.2 Daily Request Volume	27
	5.2.3 Traffic by Weekday	27
	5.2.4 Most Visited Content	28
	5.3 Error Analysis	29
	5.4 Performance Analysis	30
	5.5 Anomaly Detection Results	31
	5.6 Bot Detection Results	32
	5.7 Attack Detection Results	32
	5.7.1 Comparison with the Existing Classifiers	32
	5.7.2 Results of the Proposed Model	33
	5.7.3 Stability and Transfer to New Data	34
	5.7.4 Case Study on the Unlabelled NASA Logs	34
	5.8 Scalability	35
	5.9 Visualization Results	36
	5.10 Discussion of Results	41
	5.11 Summary of Findings	42
6	**CONCLUSION AND FUTURE WORK**	**43**
7	**REFERENCES**	**45**

LIST OF FIGURES

Figure No	Title	Page No
3.1	Overall system architecture of the Log Analytics System	12
4.1	HDFS write path in the Docker cluster	16
4.2	Execution of the Spark ETL job on the cluster	17
4.3	Run of the Spark ETL job on the full NASA logs	18
4.4	Run of the preprocessing job on the labelled data	21
5.1	Requests by hour of day	27
5.2	Daily request volume, 1 July – 31 August 1995	27
5.3	Requests by day of the week	28
5.4	Top 10 most-visited pages	28
5.5	Requests and bandwidth by file type	29
5.6	Distribution of HTTP status codes	29
5.7	Daily 404 (Not Found) error rate	30
5.8	Top 10 broken links	30
5.9	Distribution of response sizes	31
5.10	Requests per hour with the 12 anomalous hours ($\|z\| > 3$)	31
5.11	Behaviour of the three GMM components	32
5.12	Precision, recall and F1 of the eight models	33
5.13	F1 within one dataset and across datasets	34
5.14	Run time against data size with one and two cores	35
5.15	Traffic dashboard in Apache Superset	36
5.16	Anomalies (Z-score) dashboard in Apache Superset	37
5.17	Bot detection (GMM) dashboard in Apache Superset	38
5.18	Attack detection (ML) dashboard in Apache Superset	39
5.19	LogLens home page with an embedded Superset dashboard	40
5.20	LogLens 'Check a request' page running the saved model	41

LIST OF TABLES

Table No	Title	Page No
2.1	Summary of the literature reviewed	6
3.1	NASA-HTTP dataset summary	8
3.2	Fields of a Common Log Format line	8
3.3	Labelled attack datasets	9
3.4	Schema of the curated log table	10
3.5	Behavioural features per host	11
4.1	Containers in the Docker Compose cluster	15
4.2	Spark SQL reports	18
4.3	Preprocessing of the labelled attack data	20
4.4	Tuning of the Factorization Machine on the validation split	22
4.5	Apache Superset dashboards	23
4.6	Division of work among team members	24
5.1	Data processing summary	26
5.2	HTTP status code distribution	29
5.3	Response size percentiles (HTTP 200)	30
5.4	The three components of the Gaussian Mixture Model	32
5.5	Eight Spark MLlib classifiers on the same test set	33
5.6	Confusion matrix of the Factorization Machine (test set)	33
5.7	Stability over five splits and transfer across datasets (F1)	34
5.8	Hand audit of the flagged NASA URLs	35
5.9	Run time of the ETL and the eleven reports	35

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AQE	Adaptive Query Execution
BI	Business Intelligence
CLF	Common Log Format
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
F1	Harmonic mean of precision and recall
FM	Factorization Machine
FN / FP	False Negative / False Positive
GB / MB / KB	Gigabyte / Megabyte / Kilobyte
GBT	Gradient-Boosted Trees
GFS	Google File System
GMM	Gaussian Mixture Model
HDFS	Hadoop Distributed File System
HTTP	HyperText Transfer Protocol
JDBC	Java Database Connectivity
JDK	Java Development Kit
JSON	JavaScript Object Notation
KSC	Kennedy Space Center
ML / MLlib	Machine Learning / Spark Machine Learning Library
MLP	Multilayer Perceptron
NASA	National Aeronautics and Space Administration
PR-AUC	Area under the Precision–Recall curve
RDD	Resilient Distributed Dataset
RPC	Remote Procedure Call
SDK	Software Development Kit
SQL	Structured Query Language
SVM	Support-Vector Machine
TN / TP	True Negative / True Positive
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
XSS	Cross-Site Scripting

CHAPTER 1
INTRODUCTION

Every request made to a web server is written as one line in an access log: which host asked, for which page, at what time, whether the request succeeded, and how many bytes were sent back. A busy website writes millions of these lines every day. Hidden inside them are answers that matter to the people who run the site — which pages are popular, when traffic peaks, which links are broken, which clients are programs rather than people, and which requests are attempts to attack the site. In practice these logs are rarely read, because they are too large to open in a spreadsheet and too unstructured to query directly.

This project, the Log Analytics System for a Web Server, builds a complete big-data system that turns raw access logs into traffic, error and performance reports, detects unusual traffic and automated clients, classifies requests as normal or attack, and presents every result through big data visualisation tools. It runs on a real big-data stack — the Hadoop Distributed File System (HDFS) for storage, Apache Spark and Spark SQL for processing, Spark MLlib for machine learning, and Apache Superset on the Spark Thrift Server for visualisation — packaged as a cluster of ten Docker containers. The system is demonstrated on 3,461,613 real requests made to NASA's Kennedy Space Center web server in July and August 1995, and on 1.37 million labelled attack requests from two public datasets. This report covers the complete project.

1.1 Problem Statement

Web server access logs grow continuously and quickly reach sizes that conventional tools cannot handle. A single month of logs from a moderately busy server runs into hundreds of megabytes and millions of lines. Spreadsheets cannot open files of this size, single-machine scripts become slow and fragile, and the raw text has no structure that a database or reporting tool can query directly. As a result, website owners rarely see facts that are recorded in their own logs: the broken links visitors keep hitting, the hours of peak load, the bots crawling the site, or the requests that try to inject code into it.

The problem addressed in this project is therefore: how to store large web server access logs safely and process them at scale, so that traffic, error, anomaly, bot and attack questions can be answered accurately, and the answers presented through big data visualisation tools that non-technical users can read. The solution must tolerate the failure of a storage node, must never alter the original logs, must report its own data quality, and must be reproducible on an ordinary laptop.

1.2 Motivation

Access logs are one of the richest and most neglected data sources in any organisation that runs a website. Every broken link, traffic spike, crawler and attack attempt is already recorded, yet

the information remains locked in text files. Big-data platforms such as Hadoop and Spark were created precisely for this situation: data that is too large for one machine and needs to be split, processed in parallel and summarised. Building a log analytics system is a practical way to demonstrate the core ideas of big-data engineering — distributed storage, fault tolerance through replication, parallel ETL, columnar storage, partitioning, SQL on large data and distributed machine learning — on a problem whose results are easy to verify and immediately useful.

The NASA-HTTP dataset was chosen because it is real, public and unlabelled. Nobody has cleaned it in advance, so every step from parsing to reporting has to be engineered and validated by the team. It uses the Common Log Format, which Apache and Nginx servers still write today, so the same pipeline can be pointed at a modern server's logs without changes. Because the NASA logs carry no attack labels, the attack detector is trained on two public labelled datasets, which also allows the team to test honestly how well a detector transfers from one source of traffic to another.

1.3 Objectives

The objective of the project is to design and implement a distributed system that stores, cleans, analyses, mines and presents web server access logs. The specific objectives are:

- To build a reproducible Hadoop and Spark cluster using Docker, with HDFS replication so that the loss of one DataNode does not lose any data.
- To ingest the raw logs into an immutable raw zone in HDFS that serves as the audit trail of the pipeline.
- To parse every log line with Apache Spark into typed, validated columns, keeping malformed lines aside instead of silently dropping them, and to store the result as Snappy-compressed Parquet partitioned by date.
- To answer traffic, content, error and performance questions with eleven readable Spark SQL reports.
- To detect unusual hours with a contextual Z-score computed in Spark SQL.
- To separate automated clients from human visitors without labels, using a Gaussian Mixture Model in Spark MLlib on behavioural features per host.
- To classify HTTP requests as normal or attack with a Factorization Machine in Spark MLlib, compare it fairly with seven other MLlib classifiers, and test whether it generalises to data from a different source.
- To present every result only through big data visualisation tools: Apache Superset dashboards that query HDFS through the Spark Thrift Server, embedded in an application (LogLens) that can also run the saved model on any request.

1.4 Ideas

The central idea is to separate the pipeline into zones, as professional data platforms do. A raw zone holds the logs exactly as received and is never modified. A curated zone holds the

cleaned, typed data and can always be rebuilt from raw. A reports zone holds the results of the SQL reports, the anomaly detector, the bot detector and the machine-learning experiments. A models zone holds the saved attack-detection model. Each zone is written by one stage and never edited by a later one, which makes the system easy to reason about, easy to test and safe to re-run.

A second idea is to keep every number traceable. Every report is one named Spark SQL query, every machine-learning result is written to HDFS as a table, and the dashboards read those tables directly through the Spark Thrift Server, so nothing is copied out of the cluster and every chart can be traced back to a Spark job. A third idea is to measure the data before trusting a model: inspecting the labelled attack data revealed a shortcut that a model could exploit, and removing it changed which model looked best.

1.5 Solution Proposed

The proposed solution has two paths that feed one application (Fig 3.1). In the first path, the NASA logs are copied into the HDFS raw zone, where every block is stored on two DataNodes; a Spark ETL job parses each line into typed columns and writes date-partitioned Parquet; and three analyses run on the clean table: eleven Spark SQL reports, a contextual Z-score anomaly detector, and a Gaussian Mixture Model that clusters hosts by behaviour. In the second path, two labelled attack datasets are cleaned, de-duplicated and freed of a dataset bias; character n-gram features are built; a Factorization Machine is trained and compared with seven other models; and the best transferring model is tested on a dataset it has never seen and saved to HDFS. All results are registered as tables in the Spark Thrift Server, Apache Superset draws six dashboards from them, and the LogLens application embeds those dashboards and runs the saved model on requests typed by a user. The whole system starts with one command.

1.6 Challenges

Several challenges were faced while building the system. Running ten containers — a NameNode, two DataNodes, a Spark master and worker, a client, the Thrift Server, Superset and the application — requires at least 8 GB of memory for Docker. Downloading Hadoop and Spark from the throttled Apache archive took several hours on some networks, which was solved by trying mirrors first and allowing locally supplied archives. Container hostnames do not reverse-resolve, so the HDFS hostname check had to be disabled. The dataset has gaps — no requests are recorded for 29–31 July and 2 August 1995, and the server was shut down for Hurricane Erin from 1 August 14:52 to 3 August 04:36 — so charts show these gaps rather than interpolate across them.

On the machine-learning side, the labelled attack data contained a shortcut: 42.2% of attacks but only 2.5% of normal requests contained a question mark, so a model could score well without reading the request. Attacks are only 2.94% of the data, which makes accuracy misleading. On the presentation side, the course allowed only big data visualisation tools, so a hand-written web site from the first half was replaced by Apache Superset; embedding Superset in an application

required guest tokens, and Superset's rate limiter had to be configured for dashboard building. Finally, team members used both macOS and Windows, so commands, line endings, ports and memory settings had to work on both.

1.7 Scope of the Report

This report presents the complete project, which is working end to end: the Docker cluster and HDFS storage, the Spark ETL job, the eleven Spark SQL reports, Z-score anomaly detection, GMM bot detection, the attack-detection study with the Factorization Machine and seven comparison models, the Spark Thrift Server, six Apache Superset dashboards and the LogLens application, with 35 automated tests and a scalability benchmark. Chapter 2 reviews the literature, Chapter 3 describes the methodology, Chapter 4 the implementation, Chapter 5 the results and Chapter 6 the conclusions and future work. The division of work among the three team members is shown in Table 4.6.

CHAPTER 2
LITERATURE REVIEW

The design of this project rests on three bodies of work: the distributed systems research that made large-scale data processing practical, the web usage mining research that defined how server logs should be prepared and analysed, and the machine-learning research on anomaly, bot and attack detection. This chapter reviews these studies and explains how each one shaped the system.

Dean and Ghemawat introduced MapReduce [1], a programming model in which a computation is expressed as a map function and a reduce function that the framework runs in parallel across a cluster of commodity machines. MapReduce writes intermediate results to disk after every stage, which makes it slow for workloads that pass over the same data repeatedly — one of the reasons this project uses Spark instead. The storage layer underneath MapReduce was the Google File System of Ghemawat, Gobioff and Leung [2], which splits files into large chunks and replicates each chunk across several servers so that hardware failure is treated as normal. Its design was reproduced in open source as the Hadoop Distributed File System.

Shvachko et al. [3] described HDFS: a NameNode that keeps the namespace and the mapping of files to blocks, and DataNodes that store the blocks and report to the NameNode through heartbeats and block reports. This project uses HDFS 3.3.6 exactly in this form, with one NameNode, two DataNodes and a replication factor of two. Zaharia et al. [4] introduced Resilient Distributed Datasets, the abstraction behind Apache Spark, which keeps working data in memory and rebuilds lost partitions from their lineage. Armbrust et al. [5] extended Spark with Spark SQL, which adds the DataFrame API and the Catalyst optimiser. This project's ETL job is written with DataFrames and its reports and anomaly detector as plain Spark SQL. Melnik et al. [6] described Dremel, whose columnar storage format lets a query read only the fields it needs; Apache Parquet, used for every table in this project, implements the same layout. Meng et al. [18] described MLlib, Spark's distributed machine-learning library, which provides every model used in this project.

On the analysis side, Arlitt and Williamson [7] studied the access logs of six web servers, including the same NASA Kennedy Space Center server used in this project, with single-machine tools; this project repeats that analysis at scale. Cooley, Mobasher and Srivastava [8] showed that raw server logs need careful preparation — cleaning, user identification and session identification — before any meaningful analysis, and Srivastava et al. [9] defined web usage mining as preprocessing, pattern discovery and pattern analysis. Greenwald and Khanna [10] described the approximate quantile algorithm that Spark's `percentile_approx` follows. Oliner, Ganapathi and Xu [11] surveyed log analysis and named data volume and messy formats as its main obstacles.

For anomaly detection, Chandola, Banerjee and Kumar [14] surveyed statistical and machine-learning methods and showed that simple statistical scores work well when normal

behaviour can be modelled; this project's contextual Z-score compares each hour only with the same hour of other days. Xu et al. [12] detected problems in console logs with principal component analysis, and Du et al. [13] learned normal log sequences with an LSTM network in DeepLog; both target system logs rather than web traffic. For bot detection, Reynolds [15] described Gaussian Mixture Models, which model a population made of a few sub-groups of different shape and size and give each point a probability of belonging to each group; this is the model used here, validated with the silhouette coefficient of Rousseeuw [16].

For attack detection, Rendle [17] introduced Factorization Machines, which extend a linear model with a weight for every pair of features, factorised so that pairs never seen together in training still receive a weight. They come from recommender systems; the team is not aware of their use for web-attack detection on Spark. Saxe and Berlin [19] showed that character-level models can detect malicious URLs from raw strings, which motivates the character n-gram features used here. Geirhos et al. [20] described shortcut learning, where a model exploits an incidental cue in the data instead of the intended signal — exactly the problem found in the labelled attack data. The comparison models are the support-vector machine of Cortes and Vapnik [21], random forests [22] and gradient boosting [23]. Saito and Rehmsmeier [24] showed that precision–recall measures are more informative than ROC measures on imbalanced data, and McNemar's test [25], recommended by Dietterich [26] for comparing two classifiers on the same test set, is used to check whether differences between models are significant.

Table 2.1: Summary of the literature reviewed

No.	Author & Year	Method	Key Finding	Relevance to this Project
1	Dean & Ghemawat (2004)	Map and reduce functions run in parallel over a cluster	Large log jobs can be split across many machines	Disk-based and slow for repeated passes; motivates Spark
2	Ghemawat et al. (2003)	Large file chunks replicated across servers	Commodity machines can store huge data reliably	Design reproduced by HDFS
3	Shvachko et al. (2010)	NameNode index + DataNodes with replicated blocks	Open-source, fault-tolerant storage	Our raw, curated, report and model zones
4	Zaharia et al. (2012)	In-memory datasets rebuilt from lineage	Up to 20× faster than Hadoop on iterative jobs	Engine of every job
5	Armbrust et al. (2015)	DataFrames + Catalyst optimiser	SQL and code optimised together	Our reports and Z-score detector
6	Melnik et al. (2010)	Nested columnar storage	Queries read only the needed fields	Format behind our Parquet tables
7	Arlitt & Williamson (1997)	Workload study of six web servers incl. NASA	Common patterns across servers	We repeat the analysis at scale
8	Cooley et al. (1999)	Cleaning and session identification	Logs need preprocessing	Our ETL rejects and 30-min sessions

No.	Author & Year	Method	Key Finding	Relevance to this Project
11	Oliner et al. (2012)	Survey of log analysis	Volume and formats are the main obstacles	Motivates the distributed design
13	Du et al. (2017)	LSTM on system log sequences	Learns normal log behaviour	System logs, not web traffic
14	Chandola et al. (2009)	Survey of anomaly detection	Statistical scores work when normal is modelled	Contextual Z-score per hour of day
15	Reynolds (2009)	Gaussian Mixture Models	Soft clusters of different shape and size	Bot detection on 9 features
17	Rendle (2010)	Factorization Machines	Pairwise interactions in linear time	Our attack classifier
18	Meng et al. (2016)	MLlib on Spark	Distributed ML library	All 8 classifiers and the GMM
19	Saxe & Berlin (2017)	Character-level model on URLs	Raw strings reveal malicious URLs	Character n-gram features
20	Geirhos et al. (2020)	Shortcut learning	Models exploit incidental cues	Our '?' bias fix
24	Saito & Rehmsmeier (2015)	Precision–recall on imbalanced data	PR better than ROC when classes are rare	F1 and PR-AUC as metrics
25, 26	McNemar (1947); Dietterich (1998)	Paired test of two classifiers	Is the difference significant?	SVM vs FM on 5 splits

Research gap. The distributed systems papers provide general-purpose storage, processing and learning engines but no ready analysis of web logs, while the web usage mining papers define what to analyse but were developed for single machines. Attack-detection studies usually report one mixed train/test split and rarely measure shortcuts in the data or test on a second dataset, and published log tools tend to hide their calculations or require hand-written dashboards. This project combines these strands into one reproducible system: replicated HDFS storage with an immutable raw zone, Spark ETL that reports its own data quality, readable Spark SQL analytics, unsupervised bot detection, an attack detector evaluated within and across datasets after removing a measured bias, and dashboards built only with big data visualisation tools on top of HDFS.

CHAPTER 3
PROPOSED METHODOLOGY

This chapter describes the methodology of the Log Analytics System. The approach is a zoned pipeline with two paths: data is collected, stored unchanged, cleaned into a structured form, analysed with SQL and machine learning, and finally served to big data visualisation tools. Each stage reads the output of the previous one and writes a new, clearly named dataset, so that every result can be traced back to its source.

3.1 Data Collection

The main dataset is NASA-HTTP, published by the Internet Traffic Archive [30]. It contains every HTTP request made to the NASA Kennedy Space Center web server in Florida between 1 July and 31 August 1995, in two files. The collection script downloads the compressed files (about 37 MB) from a GitHub mirror, falling back to the original archive, and unpacks them into 356 MB of plain text.

Table 3.1: NASA-HTTP dataset summary

Property	Value
Source	NASA Kennedy Space Center WWW server, Internet Traffic Archive
Files	access_log_Jul95 and access_log_Aug95
Total lines (requests)	3,461,613
Distinct hosts / URLs	137,978 / 31,030
Size	about 37 MB compressed, 356 MB uncompressed
Period	1 July – 31 August 1995 (58 days with data)
Time zone	UTC−04:00 (server local time)
Known gaps	No data for 29–31 July and 2 August; server shut down for Hurricane Erin, 1 Aug 14:52 – 3 Aug 04:36
Labels	None (unsupervised analysis)

Each line follows the Common Log Format. A representative line and the meaning of each field are shown in Table 3.2.

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

Table 3.2: Fields of a Common Log Format line

Part of the line	Field	Meaning
199.72.81.55	host	The client's IP address or host name
- -	identity, user	Always empty in this dataset
[01/Jul/1995:00:00:01 -0400]	timestamp	Date and time of the request, with time zone

Part of the line	Field	Meaning
"GET /history/apollo/ HTTP/1.0"	request	Method, requested URL and protocol
200	status	HTTP status code (200 OK, 304 Not Modified, 404 Not Found …)
6245	bytes	Size of the response body; '-' when there is no body

The NASA logs carry no attack labels, so the attack detector learns from two public labelled datasets (Table 3.3), which are also copied into HDFS. All attack types are merged into one class, which makes the task supervised binary classification with two classes: 0 = normal and 1 = attack.

Table 3.3: Labelled attack datasets

Dataset	Content	Normal	Attack
HttpParams [31]	HTTP parameter values: normal, SQL injection, XSS, command injection, path traversal	19,304	11,763
Fwaf [32]	Full URLs from web traffic and attack payload lists	1,294,531	48,126
Total	Loaded and labelled	1,313,835	59,889

3.2 Data Preprocessing

3.2.1 Preprocessing of the Web Logs

Preprocessing converts each free-text log line into a structured record. A Spark job reads the raw zone line by line and applies five regular expressions: one each for the host (the first word), the timestamp (the text inside square brackets), the request (method, URL and protocol inside quotation marks), the status (a three-digit number) and the bytes (the last field). Each extracted value is then cast to a proper type: the timestamp is parsed with the pattern `dd/MMM/yyyy:HH:mm:ss Z`, the status becomes an integer and the bytes become a long integer, with a dash converted to zero.

A record is considered malformed if its timestamp cannot be parsed, its status is missing or its host is empty. Malformed lines are not silently discarded: they are written, exactly as they appeared, to a separate rejects folder, and the job prints the parse rate on every run and warns if it falls below 99%. In this dataset 3,461,612 of the 3,461,613 lines were parsed; the single reject is a truncated line that reads only *alyssa.p*.

3.2.2 Preprocessing of the Labelled Attack Data

The labelled data is prepared in seven steps by a Spark job: (1) load both datasets and (2) map every attack type to the single attack class; (3) URL-decode, lower-case and trim every request and drop empty ones; (3b) remove a dataset bias; (4) remove duplicate requests; (5) remove requests that appear with both labels; and (7) split the data 80/20 into training and test sets. Step 6 prints a summary of the prepared data. The counts after every step are given in Table 4.3.

The bias step is the most important. Inspecting the data showed that 42.2% of attack requests but only 2.5% of normal ones contain a question mark, because Fwaf's normal examples are mostly bare paths while its attacks are query strings. A model could score well simply by learning that a query string means an attack, a form of shortcut learning [20]. The fix keeps only the part after the first question mark, so that `/top.php?stuff=1 or 1=1` becomes `stuff=1 or 1=1` and the model must judge what is actually sent. After the fix the shares are 2.7% and 0.8%.

Attacks are only 2.94% of the final dataset. The training set is therefore balanced to four normal requests for every attack, so that the model sees enough attacks to learn from, while the test set keeps the real class mix and is never used for training or tuning.

3.3 Feature Engineering

3.3.1 Columns of the Curated Log Table

After cleaning, four derived columns are added to the logs. The `date` column partitions the data on disk, the `hour` and `dow` (day of week) columns support traffic and anomaly analysis, and the `ext` column — the lower-cased file extension — supports content and bot analysis. The curated schema is shown in Table 3.4.

Table 3.4: Schema of the curated log table

Column	Type	Description	Example
host	string	Client host or IP address	199.72.81.55
ts	timestamp	Request time (America/New_York)	1995-07-01 00:00:01
method	string	HTTP method	GET
endpoint	string	Requested URL path	/history/apollo/
protocol	string	HTTP protocol version	HTTP/1.0
status	integer	HTTP status code	200
bytes	long	Response size in bytes	6245
date	date (partition)	Calendar date, one folder per day	1995-07-01
hour	integer	Hour of the day, 0–23	0
dow	string	Day of the week	Saturday
ext	string	File extension of the URL	(empty for '/')

3.3.2 Behavioural Features for Bot Detection

The Common Log Format has no user-agent field, so automated clients must be recognised from behaviour alone. The requests of each host are first split into sessions, a new session starting after 30 minutes of inactivity. For every host with at least 20 requests (38,104 hosts), nine features are computed (Table 3.5). Heavy-tailed counts are transformed with $\log(1 + x)$, and all features are standardised so that no single feature dominates.

Table 3.5: Behavioural features per host

#	Feature	What it measures	Why it separates bots
1	head_ratio	Share of HEAD requests	Crawlers and link checkers use HEAD; browsers rarely do
2	med_gap	Median seconds between requests	Bots crawl at a slow, steady pace; humans burst
3	image_ratio	Share of image requests	Browsers load page images; crawlers skip them
4	n_sessions	Number of sessions	Bots return again and again
5	hits_per_session	Requests per session	Crawlers make very long visits
6	n_topdirs	Distinct top-level folders visited	Crawlers sweep the whole site
7	unique_ratio	Distinct URLs / requests	Crawlers fetch each page once
8	error_ratio	Share of 404 responses	Crawlers follow old or broken links
9	gap_cv	Std. deviation / mean of the gaps	Programs run on a timer, so timing is regular

3.3.3 Text Features for Attack Detection

Each request is split into characters, from which Spark builds character 1-, 2- and 3-grams. Each n-gram size is hashed into 4,096 buckets with HashingTF, and four counts are appended: length, special characters, digits and attack keywords (for example `union`, `<script` and `../`). This gives 12,292 sparse features, scaled to the range 0 to 1 with MaxAbsScaler so that the vectors stay sparse. The feature pipeline is fitted on the training set only.

3.4 System Architecture

The system architecture is shown in Fig 3.1. Two paths feed one application. In path 1, web log analytics, the NASA logs enter the HDFS raw zone, where every block is stored twice; the Spark ETL job cleans them into Parquet stored as 58 date folders; and three analyses run on that clean table: the Spark SQL reports, the Z-score anomaly detector and the GMM bot detector. In path 2, attack detection, the labelled datasets are preprocessed, the Factorization Machine is trained in Spark MLlib and then tested on a dataset it has never seen. At the bottom, every result stored in HDFS is exposed by the Spark Thrift Server as SQL tables over JDBC; Apache Superset draws the dashboards from those tables; and the LogLens application embeds the dashboards and runs the saved model.

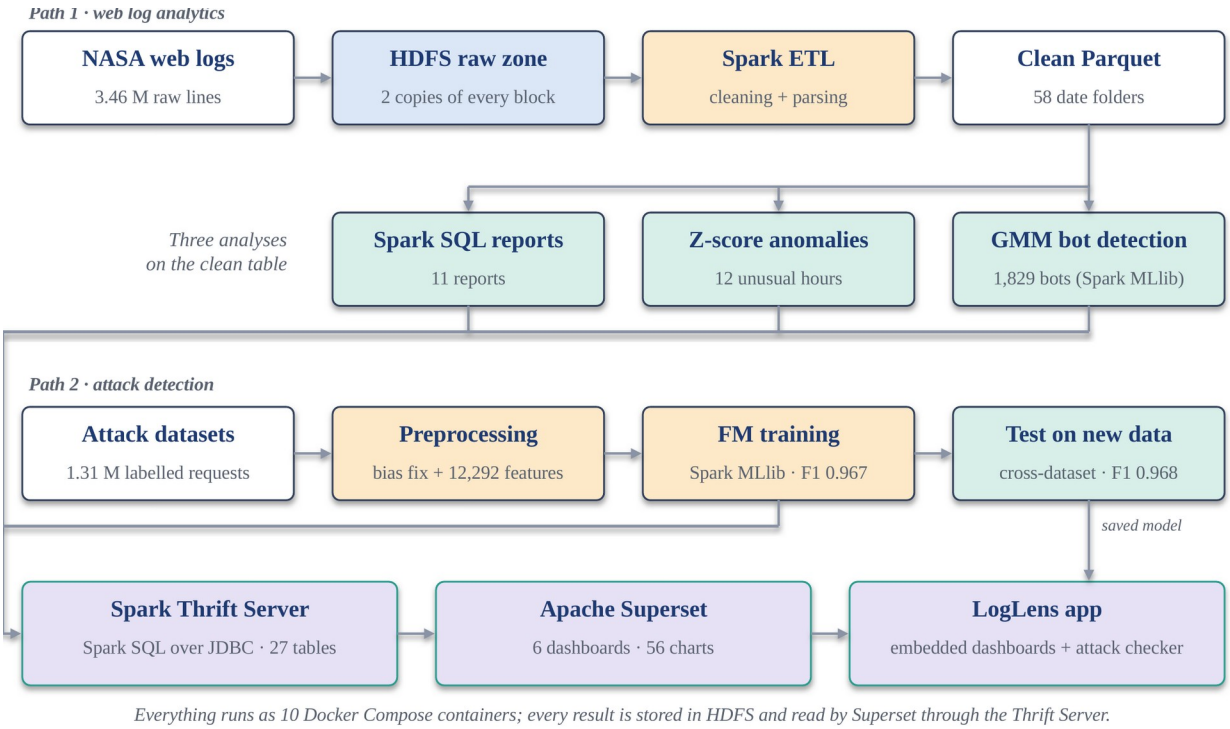


Fig 3.1: Overall system architecture of the Log Analytics System

Data moves through four HDFS zones, each written by one stage and never edited by a later one: *raw* (the files as downloaded, the audit trail), *curated* (typed Snappy Parquet, one folder per date), *reports* (Spark SQL, anomaly, bot and ML result tables) and *models* (the saved Spark ML pipeline). Because the raw zone is immutable, every result can be rebuilt from it. All components run in ten Docker containers built from two images: one with Java 11, Hadoop 3.3.6, Spark 3.5.3 and Python for every cluster role, and one with Apache Superset 4.1.1.

3.5 Analytical Models

3.5.1 Contextual Z-Score Anomaly Detection

Traffic follows a daily rhythm, so comparing every hour with the overall average would flag every busy afternoon. Requests are therefore counted for every (date, hour) slot, and each slot is compared only with the same hour of the day on all other days [14]:

$$z(d, h) = (n(d, h) - \mu(h)) / \sigma(h) \qquad (3.1)$$

where $n(d, h)$ is the number of requests on date d at hour h, and $\mu(h)$ and $\sigma(h)$ are the mean and population standard deviation of hour h across all days. A slot with $|z| > 3$ is flagged as anomalous, because under a normal distribution only about 0.3% of values fall that far from the mean. The same test is applied to daily totals. The detector needs no labels and is written in Spark SQL.

3.5.2 Gaussian Mixture Model for Bot Detection

A Gaussian Mixture Model [15] assumes that the hosts come from a mixture of K multivariate normal distributions, each with its own mean vector and full covariance matrix, and

fits them with the expectation–maximisation algorithm. Each host receives a probability of belonging to each component and is assigned to the most likely one:

$$p(x) = \Sigma_k \pi_k \cdot N(x \mid \mu_k, \Sigma_k), \quad k = 1 \dots K \qquad (3.2)$$

The model is fitted in Spark MLlib with K = 3 on the nine standardised features. The component with the highest share of HEAD requests, the slowest pacing and the fewest images is labelled *automated* from its own parameters, without any labels. A GMM was chosen over K-Means because the automated group is small and elongated: K-Means with two clusters absorbed it into a larger cluster, and with eight clusters it split it into fragments. As a held-out check, hosts that requested /robots.txt — a file read almost only by crawlers — are not used as a feature but are counted in each cluster afterwards.

3.5.3 Factorization Machine for Attack Detection

A Factorization Machine [17] extends logistic regression with a weight for every pair of features. Instead of learning each pair weight separately, it gives every feature i a small vector v_i of k numbers and uses the dot product of two vectors as the pair weight:

$$\hat{y}(x) = \sigma(w_0 + \Sigma_i w_i x_i + \Sigma_i \Sigma_{j>i} \langle v_i , v_j \rangle x_i x_j) \qquad (3.3)$$

where σ is the logistic function, w_0 the bias, w_i the weight of feature i and $v_i \in \mathbb{R}^k$ its factor vector. Pairs of n-grams that never appear together in training still receive a sensible weight through their vectors, and the pairwise term can be computed in O(k·n) time, linear in the number of non-zero features. This suits attack detection, because attacks are combinations — a quote together with the word *or* and a comment marker — rather than single tokens. Spark MLlib's FMClassifier is used, with settings chosen on a validation split: factor size k = 8, step size 0.2, 100 iterations and L2 regularisation of 10^{-4}.

3.5.4 Comparison Classifiers

Seven other Spark MLlib classifiers are trained on exactly the same features and data: a linear support-vector machine [21] and logistic regression (100 iterations, L2 10^{-4}), multinomial Naive Bayes, a decision tree (depth 10), a random forest [22] (30 trees, depth 10), gradient-boosted trees [23] (15 rounds, depth 5) and a multilayer perceptron with layers 388–32–2. Spark's perceptron needs dense input, so it uses a 388-feature version of the same n-grams.

3.6 Evaluation Metrics

The data pipeline is evaluated with data-quality and system metrics, and the models with classification and clustering metrics:

- **Parse rate and reconciliation** — the share of raw lines that become valid records, and curated rows + rejects = raw lines.

- **Replication health and fault tolerance** — `hdfs fsck` must report two live replicas for every block, and data must stay readable when one DataNode is stopped.
- **Partition pruning** — a query on one date must read only that date's folder rather than all 58.
- **Precision, recall and F1 of the attack class** — precision = TP / (TP + FP) is the share of alarms that are real attacks; recall = TP / (TP + FN) is the share of attacks caught; F1 = 2·P·R / (P + R) is their harmonic mean. Because 97% of test requests are normal, a model that never detects an attack would still reach 97% accuracy, so F1 is the main metric, together with the confusion matrix and PR-AUC [24].
- **Stability and significance** — mean and standard deviation of F1 over five random splits, and McNemar's test [25], [26] on the requests where two models disagree.
- **Transfer** — F1 when training on one dataset and testing on the other.
- **Clustering quality** — the silhouette coefficient [16] of the GMM, and the lift of known crawlers (robots.txt requesters) in the automated cluster.
- **Traceability** — every chart must be a Spark SQL query on an HDFS table.

3.7 Visualization Techniques

The course allows only big data visualisation tools for the presentation layer, so all charts are produced by Apache Superset [27], an open-source business-intelligence platform comparable to Power BI. Superset connects to the Spark Thrift Server [28], which exposes Spark SQL over JDBC; every chart is therefore a Spark SQL query that runs on the cluster against tables stored in HDFS, and nothing is copied out of the cluster. The visualisations used are line and area charts for requests over time and the daily 404 rate; bar charts for traffic by hour and weekday, model F1 scores and stability; pie and donut charts for the status mix and bot share; tables with search and sorting for broken links, anomalies, automated hosts and model metrics; big-number tiles for headline figures; and native filters, for example to switch the confusion matrix between models. SQL Lab allows any Spark SQL query to be run live.

The dashboards are shown inside LogLens, an application that provides navigation and one page per analysis. LogLens draws no charts itself: each page embeds a Superset dashboard through Superset's embedded mode with a short-lived guest token, and a *Check a request* page runs the saved Spark ML model on any URL typed by a user.

CHAPTER 4
IMPLEMENTATION

The system was implemented as a set of small, single-purpose components, each owned by one team member, following the two paths of Fig 3.1. This chapter describes the cluster, the ingestion, the Spark jobs of both paths, the serving layer and the tests. Every stage is one `make` target, and a single script starts the cluster and runs only the stages whose results are missing.

4.1 Environment and Cluster Setup

The cluster is defined in Docker [29]. One image, built from the `eclipse-temurin:11-jdk-jammy` base (Ubuntu 22.04 with Java 11), installs Hadoop 3.3.6, Spark 3.5.3 and Python 3 with the libraries the jobs and the application need; a second image extends the official Apache Superset 4.1.1 image with the PyHive driver that talks to the Spark Thrift Server. Both images run on Intel and Apple Silicon machines. An entrypoint script gives each container its role: the NameNode formats HDFS only on its first start, the DataNodes and Spark worker wait until the NameNode or master is reachable, the Thrift Server starts once HDFS is ready, and the client container stays running for commands. Docker Compose starts the containers listed in Table 4.1; each DataNode has its own storage volume, so the two copies of a block are physically separate.

Table 4.1: Containers in the Docker Compose cluster

Container	Role	Resources / ports
namenode	HDFS NameNode: namespace and block locations	9870 (web UI), 8020 (RPC)
datanode1, datanode2	HDFS DataNodes: store the blocks, each on its own volume	replication 2, 128 MB blocks
spark-master	Spark standalone master: allocates executors	7077, 8080 (UI)
spark-worker	Spark worker: runs the executors	3 cores, 3 GB
client	Runs hdfs commands and spark-submit	mounts code and data
thrift	Spark Thrift Server: Spark SQL over JDBC, Derby metastore	10000; 1 core, 1 GB
superset	Apache Superset: dashboards and SQL Lab	8088
app	LogLens application (Flask) with a local Spark session	3080
spark-history (optional)	Spark history server for finished jobs	18080

HDFS is configured through two files. `core-site.xml` sets `fs.defaultFS` to `hdfs://namenode:8020`, so every service finds the file system at the same address. `hdfs-site.xml` sets the replication factor to 2, disables the DataNode hostname check (container names do not reverse-resolve) and disables permission checks for this single-user teaching cluster. Fig 4.1 shows how a file is written: the client asks the NameNode where each block should go, writes the block to the first DataNode, and that DataNode forwards a copy to the second.

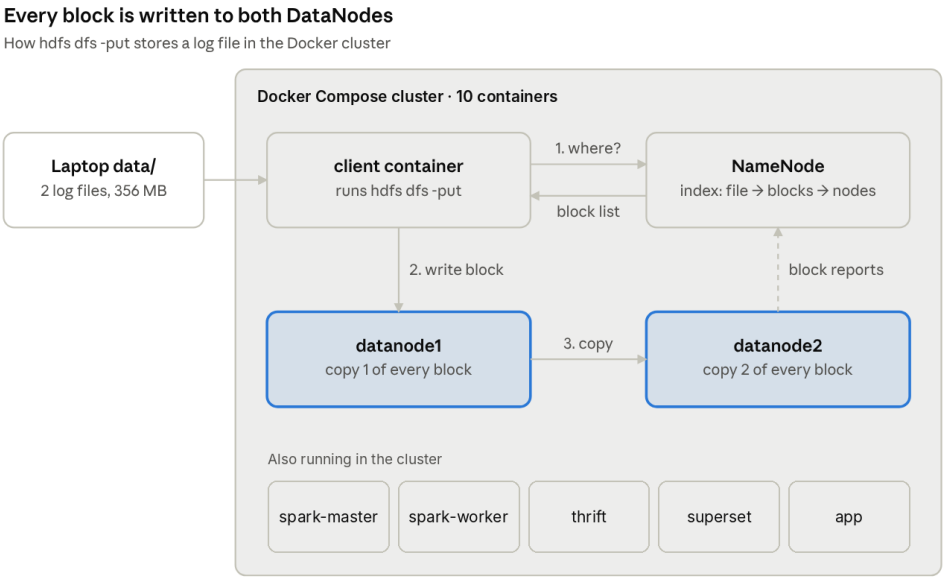


Fig 4.1: HDFS write path in the Docker cluster

4.2 Data Acquisition and Ingestion into HDFS

The download script fetches the compressed files, writes each to a temporary `.part` file and renames it only when complete, so an interrupted download never looks finished. The ingestion step creates the HDFS zones and copies the raw logs and the labelled attack datasets into them:

```
hdfs dfs -mkdir -p /data/weblogs/raw /data/weblogs/curated \
                   /data/weblogs/reports /data/attacks/raw /models
hdfs dfs -put -f /workspace/data/access_log_* /data/weblogs/raw/
hdfs dfs -put -f /workspace/data/attacks/* /data/attacks/raw/
hdfs fsck /data/weblogs/raw -files -blocks -locations
```

With the default block size of 128 MB, the 196 MB July file is stored as two blocks and the 160 MB August file as two blocks; with a replication factor of 2, these four blocks are stored as eight block copies across the two DataNodes. The raw zone is never modified after ingestion.

4.3 Data Cleaning and Preprocessing using Apache Spark

The ETL job (`jobs/etl.py`) is submitted to the cluster with `spark-submit`. Each field is extracted with a regular expression and converted to a proper type, malformed records are separated, and the clean data is written as date-partitioned, Snappy-compressed Parquet:

```
raw = spark.read.text("hdfs://namenode:8020/data/weblogs/raw")
parsed = raw.select(
    F.regexp_extract("value", HOST, 1).alias("host"),
    F.regexp_extract("value", TS, 1).alias("ts_str"),
    F.regexp_extract("value", REQ, 1).alias("method"),
    F.regexp_extract("value", REQ, 2).alias("endpoint"),
    F.regexp_extract("value", REQ, 3).alias("protocol"),
    F.regexp_extract("value", STATUS, 1).cast(IntegerType()).alias("status"),
    F.regexp_extract("value", BYTES, 1).alias("bytes_str"))
  .withColumn("bytes", F.when(F.col("bytes_str") == "-", F.lit(0))
                       .otherwise(F.col("bytes_str").cast(LongType())))
  .withColumn("ts", F.to_timestamp("ts_str", "dd/MMM/yyyy:HH:mm:ss Z"))
bad   = parsed.filter(F.col("ts").isNull() | F.col("status").isNull() | (F.col("host")
== ""))
clean = parsed.subtract(bad).withColumn("date", F.to_date("ts")) ...
clean.write.mode("overwrite").option("compression", "snappy") \
     .partitionBy("date").parquet(CURATED)
```

Spark evaluates these transformations lazily: the regular expressions and casts run only when an action such as `count()` or `write` needs a result, so Spark plans the whole chain at once. Fig 4.2 shows how the job is executed: the driver in the client container builds the plan and splits it into tasks, the master starts an executor on the worker, and the executor runs the tasks in parallel, each reading one HDFS block and writing its part of the Parquet output. `spark-defaults.conf` sets 2 GB of driver and executor memory, 16 shuffle partitions, adaptive query execution and the `America/New_York` session time zone. Fig 4.3 shows a real run of the job on the full dataset.

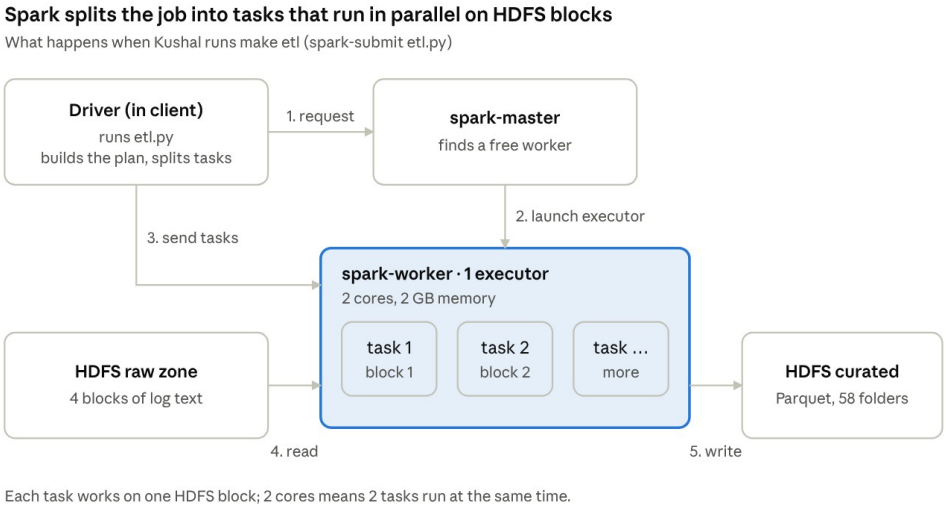


Fig 4.2: Execution of the Spark ETL job on the cluster

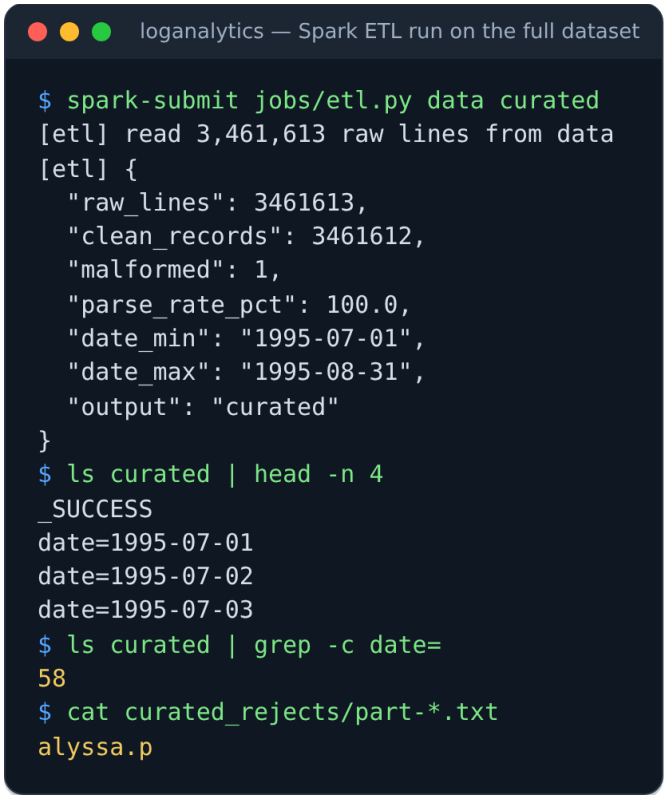


Fig 4.3: Run of the Spark ETL job on the full NASA logs

4.4 Analytical Reports using Spark SQL

The reports job (`jobs/reports.py`) registers the curated Parquet data as a SQL view named `access_logs`, caches it in memory and runs eleven SQL queries. Each result is written back to HDFS as a small Parquet table under `/data/weblogs/reports` (Table 4.2). Tables that rank items are capped, because a report is a summary rather than a second copy of the log.

Table 4.2: Spark SQL reports

#	Report	Question answered	Main SQL technique
1	by_hour	Which hour of the day is busiest?	GROUP BY hour, COUNT(DISTINCT host)
2	daily	Requests and visitors per day	GROUP BY date
3	by_weekday	Which weekday is busiest?	GROUP BY dow
4	top_urls	Most visited pages (images excluded)	WHERE status = 200 AND ext NOT IN (…), top 300
5	by_content_type	Requests and GB per file type	CASE, SUM(bytes), top 40
6	status_distribution	Share of each status code	Sub-query for the total, percentage
7	errors_daily	404 and 5xx rates per day	SUM(CASE WHEN status = 404 …)
8	broken_links	Missing URLs by hits and visitors	WHERE status = 404, COUNT(DISTINCT host)
9	size_percentiles	Typical and largest response sizes	percentile_approx
10	size_buckets	Responses per size range	CASE WHEN bytes < 1024 …

#	Report	Question answered	Main SQL technique
11	heaviest_hosts	Which clients request the most	GROUP BY host ORDER BY hits DESC

Two of the queries are shown below as examples.

```
SELECT hour, COUNT(*) AS hits, COUNT(DISTINCT host) AS visitors,
       ROUND(SUM(bytes)/1024/1024/1024, 2) AS gb
FROM access_logs GROUP BY hour ORDER BY hour;

SELECT percentile_approx(bytes, 0.50) AS p50, percentile_approx(bytes, 0.99) AS p99,
       MAX(bytes) AS max_bytes, ROUND(AVG(bytes), 0) AS mean_bytes
FROM access_logs WHERE status = 200 AND bytes > 0;
```

4.5 Anomaly Detection with Spark SQL

The anomaly job (`jobs/anomalies.py`) implements Equation 3.1 in Spark SQL. It counts requests per date and hour, computes the mean and population standard deviation of every hour of the day across all dates, joins this baseline back to the counts, and keeps the slots whose absolute Z-score exceeds the threshold of 3. Outage hours have no rows, so they appear as gaps rather than as drops. The hourly and daily results are written to HDFS as report tables.

```
WITH hourly   AS (SELECT date, hour, COUNT(*) AS hits FROM access_logs GROUP BY date,
hour),
     baseline AS (SELECT hour, AVG(hits) AS mean_hits, STDDEV_POP(hits) AS std_hits
                  FROM hourly GROUP BY hour),
     scored   AS (SELECT h.date, h.hour, h.hits, b.mean_hits AS normal_hits,
                         (h.hits - b.mean_hits) / b.std_hits AS z
                  FROM hourly h JOIN baseline b ON h.hour = b.hour)
SELECT * FROM scored WHERE ABS(z) > 3 ORDER BY ABS(z) DESC;
```

4.6 Bot Detection with Spark MLlib

The bot job (`jobs/bot_detection.py`) builds sessions with a Spark window function ordered by time per host: the gap to the previous request is computed with `lag`, a new session starts when the gap exceeds 30 minutes, and a running sum gives the session number. The nine features of Table 3.5 are then aggregated per host, skewed counts are log-transformed, and the features are assembled and standardised before the GMM is fitted:

```
f = hosts.withColumn(c, F.log1p(F.col(c)))            # for the skewed counts
asm  = VectorAssembler(inputCols=FEATURES, outputCol="raw_features").transform(f)
data = StandardScaler(inputCol="raw_features", outputCol="features",
                      withMean=True, withStd=True).fit(asm).transform(asm)
model = GaussianMixture(k=3, seed=42, maxIter=80, featuresCol="features").fit(data)
sil = ClusteringEvaluator(metricName="silhouette").evaluate(model.transform(data))
```

The job names each component from its own mean values, writes every host with its cluster and raw features (`host_clusters`), a per-cluster profile (`bot_clusters`) and a summary with the robots.txt check (`bot_summary`) to HDFS.

4.7 Attack Detection Pipeline with Spark MLlib

4.7.1 Preprocessing Job

The preprocessing job (`jobs/ml_preprocess.py`) reads both labelled datasets from HDFS and performs the steps described in Section 3.2.2, printing the row count after each step (Table 4.3, Fig 4.4). The prepared training and test sets are written back to HDFS as Parquet.

Table 4.3: Preprocessing of the labelled attack data

Step	Operation	Rows
1–2	Load both datasets, map all attack types to class 1	1,373,724
3	URL-decode, lower-case, trim, drop 67 empty rows	1,373,657
3b	Bias fix: keep only the part after '?'	—
4	Remove 66,987 duplicate requests	1,306,518 distinct
5	Remove 501 requests with both labels	1,306,017
6	Final dataset: 1,267,605 normal (97.06%), 38,412 attack (2.94%)	1,306,017
7	Training set: all training attacks + 4 normal per attack	153,847
7	Test set: real class mix, never used for training	261,076

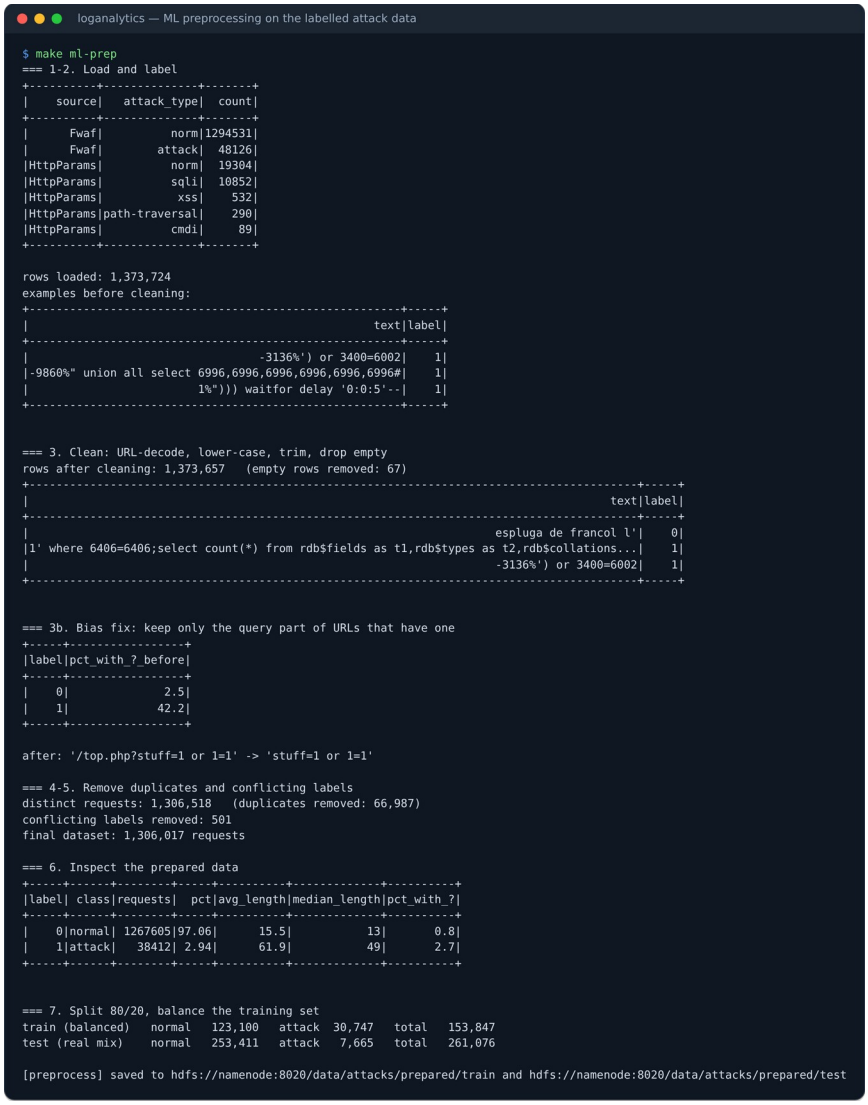


Fig 4.4: Run of the preprocessing job on the labelled data

4.7.2 Feature Pipeline and Training

The training job (`jobs/ml_train.py`) builds the features of Section 3.3.3 as a Spark ML Pipeline and trains the eight models on the same training set:

```
df = df.withColumn("chars", F.expr("filter(split(text, ''), c -> c != '')"))
feats = Pipeline(stages=[NGram(n=2, inputCol="chars", outputCol="bi"),
                         NGram(n=3, inputCol="chars", outputCol="tri"),
                         HashingTF(inputCol="chars", outputCol="f1", numFeatures=4096),
                         HashingTF(inputCol="bi",    outputCol="f2", numFeatures=4096),
                         HashingTF(inputCol="tri",   outputCol="f3", numFeatures=4096),
                         VectorAssembler(inputCols=["f1", "f2", "f3", "counts"],
outputCol="raw"),
                         MaxAbsScaler(inputCol="raw", outputCol="features")])
fm = FMClassifier(factorSize=8, stepSize=0.2, maxIter=100, regParam=1e-4, seed=42)
```

For every model the job records the confusion matrix, precision, recall, F1 and training time, and writes them to HDFS as JSON and as Parquet tables for the dashboards.

4.7.3 Hyper-parameter Tuning

The settings of the Factorization Machine were chosen by `jobs/ml_tune.py` on a validation split carved out of the training set (122,910 fitting and 30,937 validation requests); the test set is never read during tuning. Table 4.4 shows the grid. A factor size of 8 and a step size of 0.2 gave the best validation F1.

Table 4.4: Tuning of the Factorization Machine on the validation split

Factor size k	Step size	Validation F1	Training time (s)
4	0.05	0.9836	31.4
4	0.2	0.9861	31.0
8	0.05	0.9860	50.0
8	0.2	0.9882	48.9
16	0.05	0.9865	84.5
16	0.2	0.9849	87.7

4.7.4 Stability, Transfer and Model Saving

Two further jobs test whether the results hold. `jobs/ml_robustness.py` repeats the whole experiment — split, balancing, features and training — for the three strongest models with five different random seeds and runs McNemar's test on every split. `jobs/ml_crossdataset.py` trains on one dataset and tests on the other, in both directions, with five seeds. Finally, `jobs/ml_save_model.py` refits the complete pipeline (n-grams, hashing, scaling and the Factorization Machine) and saves it to `hdfs:///models/fm_attack`, reloads it and checks that it reproduces the test F1; `jobs/publish_ml_tables.py` writes all ML results as Parquet tables for Superset.

4.8 Serving Layer: Spark Thrift Server and Apache Superset

The Spark Thrift Server runs in its own container with one core and 1 GB, started with `spark-submit --class org.apache.spark.sql.hive.thriftserver.HiveThriftServer2`. Its table definitions are kept in a Derby metastore on a Docker volume. A SQL script (`sql/bi_tables.sql`), run with beeline, registers every HDFS folder as an external Spark table in a database named `weblogs` — 27 tables in total: the curated `access_logs` table (whose date partitions are discovered with `MSCK REPAIR TABLE`), the eleven reports, the anomaly and bot tables, and the ML result tables:

```
CREATE DATABASE IF NOT EXISTS weblogs;
CREATE TABLE IF NOT EXISTS weblogs.access_logs USING parquet
  LOCATION '${hivevar:BASE}/curated';
MSCK REPAIR TABLE weblogs.access_logs;
CREATE TABLE IF NOT EXISTS weblogs.by_hour USING parquet
  LOCATION '${hivevar:BASE}/reports/by_hour';
```

Apache Superset connects to the Thrift Server with the URI `hive://thrift:10000/weblogs`. A build script (`superset/build_dashboards.py`) creates the database connection, 32 datasets, 56 charts and six dashboards through Superset's REST API (Table 4.5), so the whole presentation layer is reproducible from code. Results are cached for an hour, because the tables change only when the pipeline runs. Each dashboard is registered for embedding so that the application can display it.

Table 4.5: Apache Superset dashboards

#	Dashboard	Main content
0	Home	Headline numbers, requests per day, status mix, requests by hour, model F1
1	Traffic	Requests and visitors per day, by hour and weekday, top pages, content types, heaviest hosts
2	Errors & performance	Status codes, daily 404 rate, broken links, response-size percentiles and buckets
3	Anomalies (Z-score)	Requests per hour over both months, unusual hours against their normal level, anomaly table
4	Bot detection (GMM)	Hosts and requests by segment, behaviour of the three components, automated hosts
5	Attack detection (ML)	F1 of the eight models, precision and recall, confusion matrix with model filter, stability, McNemar and transfer

4.9 LogLens Application

LogLens is a Python Flask application running in its own container on port 3080. It provides navigation and the pages Dashboard, Check a request, Traffic, Errors, Anomalies, Bots, Model performance, Data & pipeline and About. Each analysis page embeds one Superset dashboard with the Superset embedded SDK: the browser asks the application for a guest token, the application requests it from Superset's API with its service account, and the token allows viewing only that one dashboard, so users never need a Superset login. The headline numbers on the home page — requests, days, hosts, error rate, unusual hours, automated hosts and the model F1 — are computed by a local Spark session directly from the HDFS tables.

The *Check a request* page loads the saved Spark ML pipeline from HDFS once at start-up. A typed URL is normalised exactly as in training (URL-decoding, lower-casing, keeping only the part after '?'), scored by Spark in about 0.7 seconds, and shown with its attack probability, the normalised text, the counts and the attack signals found, such as SQL keywords or a script tag. Pages render with dashes while Spark is still computing the numbers after start-up, so the application never returns an error.

4.10 Verification and Testing

Each property claimed for the system was verified with a direct test, and a suite of 35 automated tests (`pytest`) runs every Spark job unchanged on a small sample with known answers:

- **Replication:** `hdfs fsck` reported every block with two live replicas on two DataNodes and an overall status of HEALTHY; with `datanode2` stopped, all data stayed readable.
- **Record reconciliation:** the ETL read 3,461,613 lines and wrote 3,461,612 curated rows plus 1 reject.
- **Partition pruning:** a query on `WHERE date = '1995-07-04'` read a single date folder instead of all 58.
- **Pipeline tests:** exact parsing of hand-written log lines, the malformed line kept as a reject, report totals that add up, and an injected traffic burst that the Z-score must find.
- **ML tests:** crawlers the GMM must separate, no request present in both training and test sets, valid confusion matrices, and a saved model that reloads and scores the test set.
- **Serving tests:** a real Spark Thrift Server that must serve every dashboard table, and every page of the application, including before Spark has finished computing.

4.11 Division of Work

The work was divided among the three team members by stage of the architecture, as shown in Table 4.6. Each member owns the files of their stages and hands a clearly defined dataset to the next; the comparison study, the tests and the documentation were done together.

Table 4.6: Division of work among team members

Member	Modules	Main files	Output handed on
Kriti Singh RA2411028010081	Docker cluster, HDFS storage and ingestion; GMM bot detection	docker/, docker-compose.yml, conf/core-site.xml, conf/hdfs-site.xml, scripts/get_data.sh, jobs/bot_detection.py	Replicated raw zone; host clusters and bot summary
Kushal Mohan RA2411028010082	Spark ETL; preprocessing of the labelled data; Factorization Machine training	conf/spark-defaults.conf, jobs/etl.py, jobs/ml_preprocess.py, jobs/ml_train.py, jobs/ml_tune.py, jobs/ml_save_model.py	Curated Parquet; train/test sets; saved FM model
Gargi Singh RA2411028010102	Spark SQL reports; Z-score anomalies; test on new data; Thrift Server, Superset and LogLens	jobs/reports.py, jobs/anomalies.py, jobs/ml_crossdataset.py, sql/bi_tables.sql, superset/, app/	11 reports, anomalies, transfer results, 6 dashboards, LogLens
All three	Model comparison, stability tests, automated tests, report and paper	jobs/ml_robustness.py, jobs/publish_ml_tables.py, tests/, docs/	Comparison tables, 35 tests, documentation

4.12 System Summary

The implemented system takes the public log files and labelled datasets and, with a handful of commands (`make data build up all ml bi`), produces a replicated raw archive in HDFS, a clean date-partitioned Parquet table, eleven report tables, anomaly and bot tables, a fully evaluated and saved attack-detection model, 27 SQL tables in the Spark Thrift Server, six Superset dashboards and the LogLens application. Every stage is reproducible, every number is traceable to a Spark job, and the original data is never modified. The same jobs also run in a Google Colab notebook.

CHAPTER 5
RESULTS AND DISCUSSIONS

This chapter presents the results produced by the complete system: data processing, exploratory analysis of traffic, errors and performance, anomaly detection, bot detection, attack detection, scalability and the visualisation layer. All numbers are computed by the Spark jobs described in Chapter 4 on the Docker cluster.

5.1 Data Processing Results

Table 5.1 summarises how the data changed size and form as it moved through the pipeline. Of the 3,461,613 raw lines, 3,461,612 were parsed into valid records — a parse rate of 99.99997% — and the single truncated line was kept as a reject. The curated Parquet table is 8.6 times smaller than the raw text.

Table 5.1: Data processing summary

Stage	Form	Size / count
Downloaded	2 compressed .gz files	about 37 MB
Raw zone (HDFS)	2 text files, 4 blocks × 2 replicas	356 MB; 3,461,613 lines
Curated zone (HDFS)	Snappy Parquet, 11 typed columns, partitioned by date	41.5 MB; 3,461,612 rows in 58 date folders; 1 reject
Reports zone (HDFS)	11 reports + anomaly, bot and ML tables	1 to 300 rows each
Models zone (HDFS)	Saved Spark ML pipeline (FM)	1 model
Totals	Distinct hosts / URLs / data transferred	137,978 / 31,030 / 61.02 GB

5.2 Exploratory Data Analysis Results

5.2.1 Traffic by Hour of Day

Fig 5.1 shows the number of requests in each hour of the day, in the server's local time. Traffic rises steeply from 66,540 requests at 06:00 to over 210,000 by 11:00, stays above 210,000 until 16:00 and peaks at 15:00 with 230,665 requests. The quietest hour is 04:00 with 58,990 requests, a peak-to-trough ratio of 3.9×, which reflects working hours in the United States.

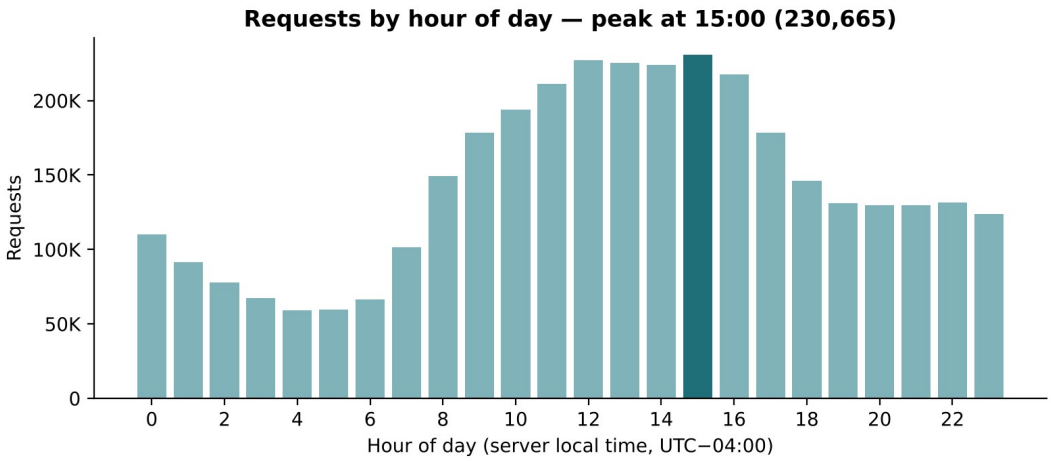


Fig 5.1: Requests by hour of day

5.2.2 Daily Request Volume

Fig 5.2 shows the daily request volume. The median day has 59,273 requests. The busiest day by far is 13 July with 134,203 requests — the launch day of Space Shuttle mission STS-70 — and the quietest full days fall on weekends. The shaded band marks the missing and partial days around the Hurricane Erin shutdown; the chart shows this gap rather than drawing a line across it.

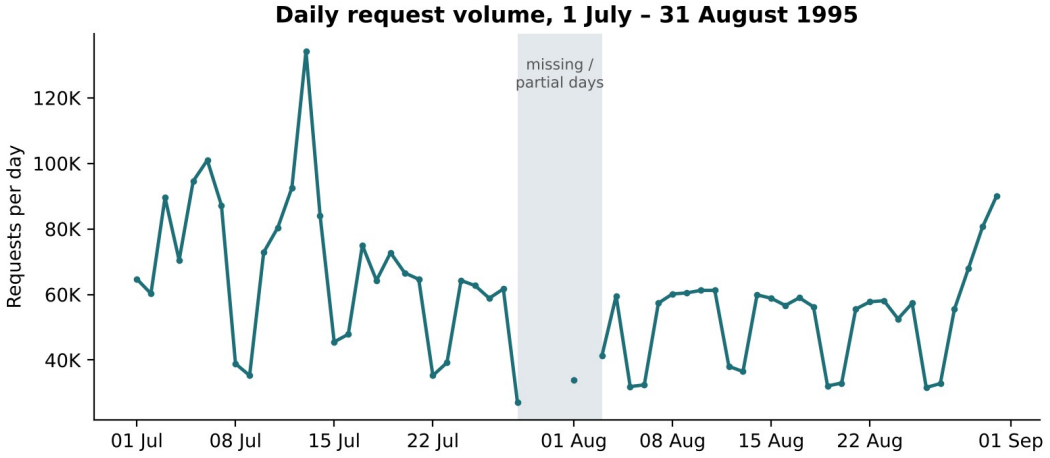


Fig 5.2: Daily request volume, 1 July – 31 August 1995

5.2.3 Traffic by Weekday

Fig 5.3 shows requests by day of the week. Thursday is the busiest day with 667,737 requests, while Saturday (318,046) and Sunday (317,276) receive less than half of that, confirming that the site was used mainly during the working week.

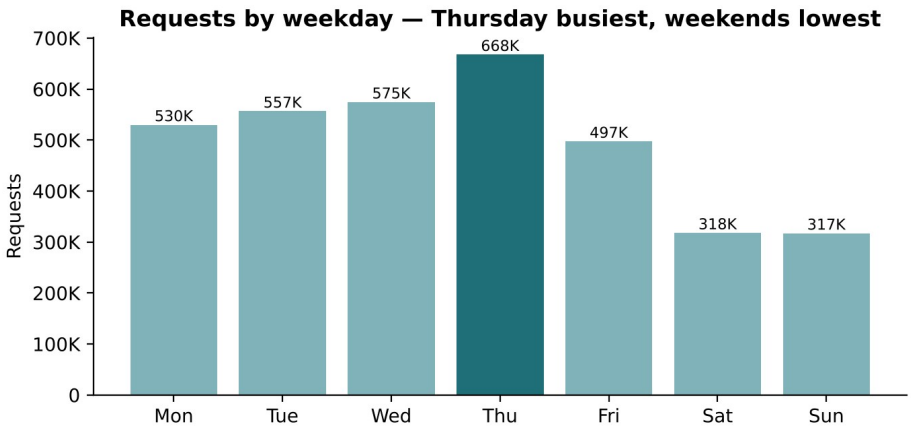


Fig 5.3: Requests by day of the week

5.2.4 Most Visited Content

Fig 5.4 lists the ten most-visited pages among successful requests, excluding images. The Kennedy Space Center home page `/ksc.html` is the most visited with 79,993 hits, followed by the shuttle countdown page (59,177) and the site root (56,875). Fig 5.5 compares requests and bytes per file type: GIF images account for 1,989,529 requests and 22.52 GB, while MPEG video files are only about 1.3% of requests but transfer 18.12 GB, nearly 30% of all bytes.

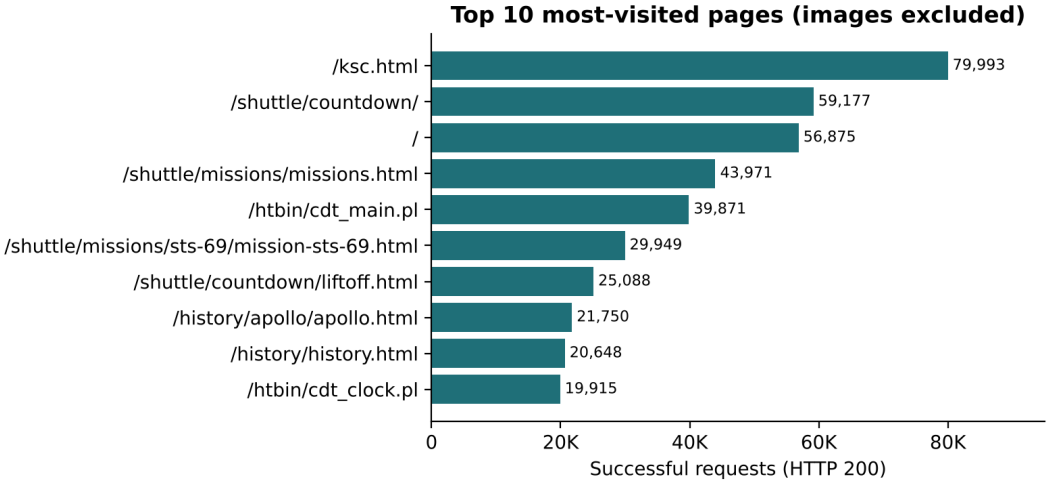


Fig 5.4: Top 10 most-visited pages

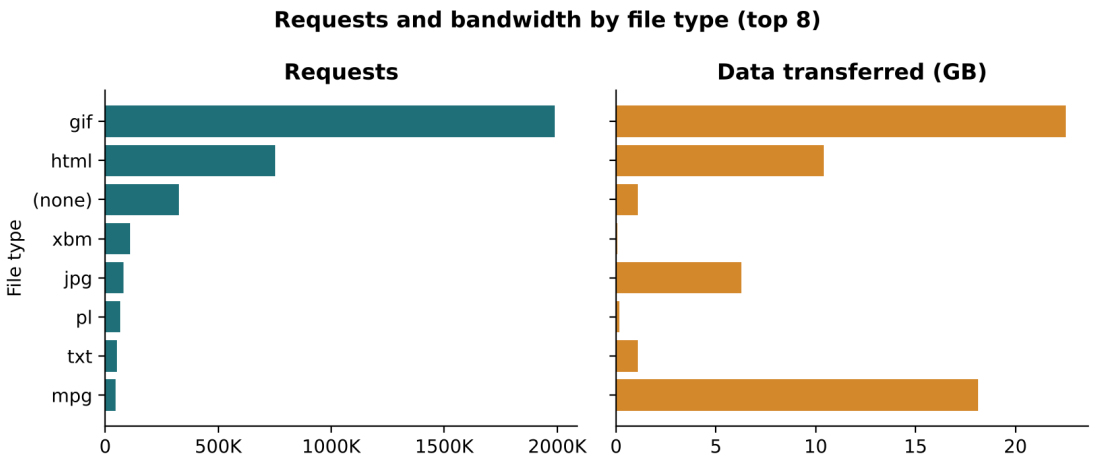


Fig 5.5: Requests and bandwidth by file type

5.3 Error Analysis

Fig 5.6 and Table 5.2 show the distribution of HTTP status codes. 89.57% of requests completed successfully (200) and 7.71% were answered with 304 Not Modified. 20,899 requests (0.60%) returned 404 Not Found, and server errors were very rare, with 106 responses in the 5xx range.

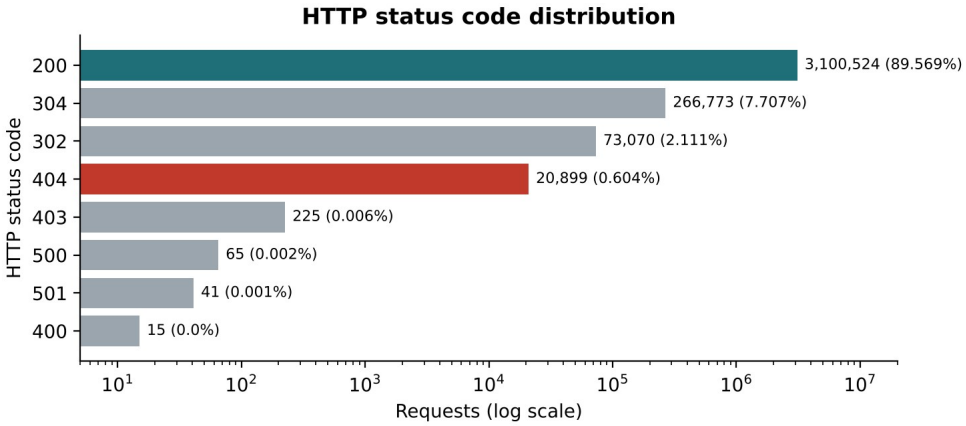


Fig 5.6: Distribution of HTTP status codes

Table 5.2: HTTP status code distribution

Status	Meaning	Requests	Share
200	OK	3,100,524	89.569%
304	Not Modified	266,773	7.707%
302	Found (redirect)	73,070	2.111%
404	Not Found	20,899	0.604%
403	Forbidden	225	0.006%
500	Internal Server Error	65	0.002%
501	Not Implemented	41	0.001%
400	Bad Request	15	< 0.001%

Fig 5.7 shows the daily 404 rate; on half of the days it lies between 0.51% and 0.72%, and the highest rates fall on weekends, when total traffic is low but requests for the same broken files continue. Fig 5.8 ranks the broken links: the 20,899 errors are spread across 3,544 URLs, but `/pub/winvn/readme.txt` (2,004 failures) and `/pub/winvn/release.txt` (1,732) alone account for 18% of all 404s.

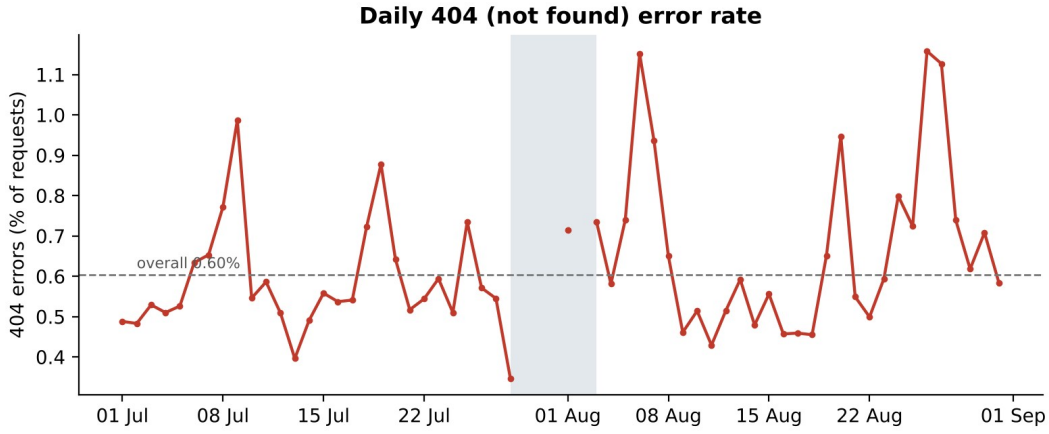


Fig 5.7: Daily 404 (Not Found) error rate

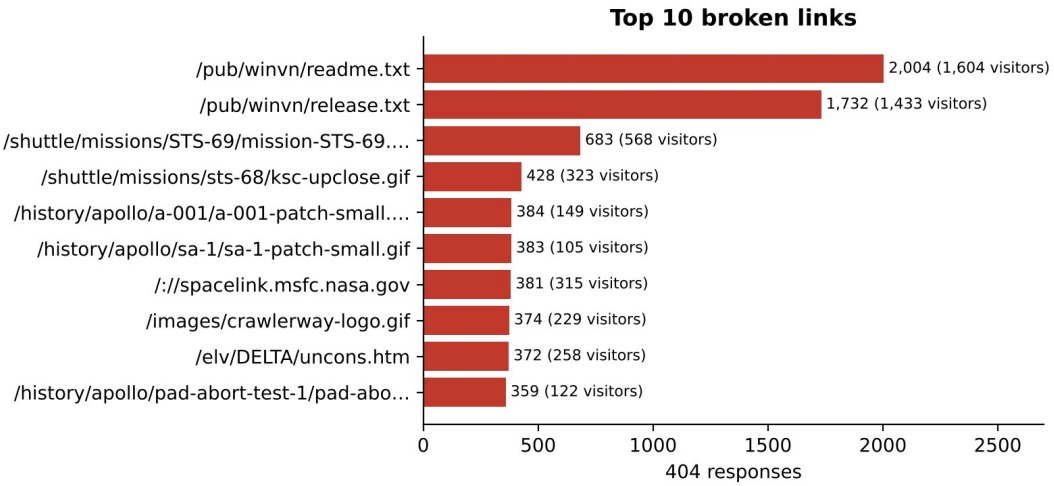


Fig 5.8: Top 10 broken links

5.4 Performance Analysis

The NASA logs do not record response times, so performance is measured by response size. Table 5.3 shows the percentiles of successful response sizes, computed with `percentile_approx`. The median response is only 4,179 bytes, but the 99th percentile is 283,389 bytes and the largest response 6.5 MB. Fig 5.9 groups responses by size; only 4,633 exceeded 1 MB.

Table 5.3: Response size percentiles (HTTP 200)

Statistic	Bytes	Approx.
Median (p50)	4,179	4.1 KB

Statistic	Bytes	Approx.
90th percentile	46,573	45.5 KB
95th percentile	83,915	81.9 KB
99th percentile	283,389	276.7 KB
Maximum	6,823,936	6.5 MB
Mean	21,222	20.7 KB

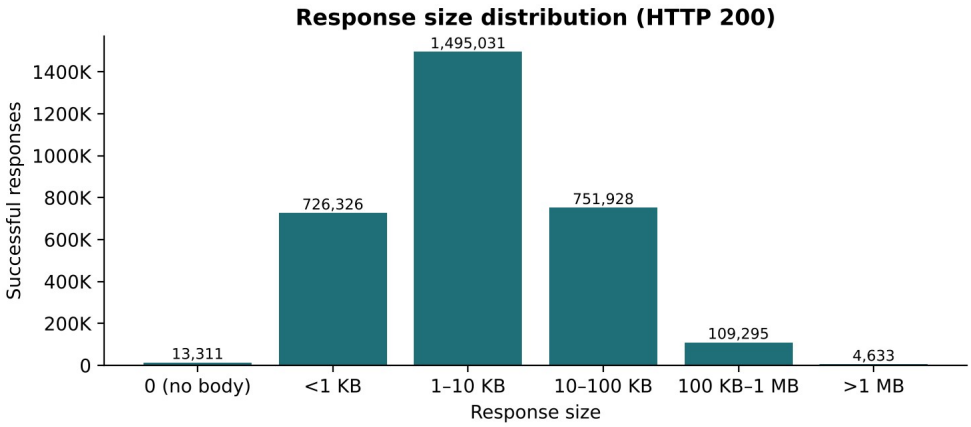


Fig 5.9: Distribution of response sizes

5.5 Anomaly Detection Results

The contextual Z-score flags 12 of the 1,369 date–hour slots (Fig 5.10). Eleven of them fall on the morning of 13 July 1995. The strongest is 09:00–10:00 that day: 14,926 requests against a normal 3,080 for that hour (σ = 2,006), a Z-score of 5.90 — the hour of the STS-70 Space Shuttle launch at 9:41 a.m. On daily totals only 13 July is flagged (z = 3.62). The twelfth hour is 31 August 04:00 (z = 3.05). The detector therefore recovered a real-world event from the traffic alone, with no labels.

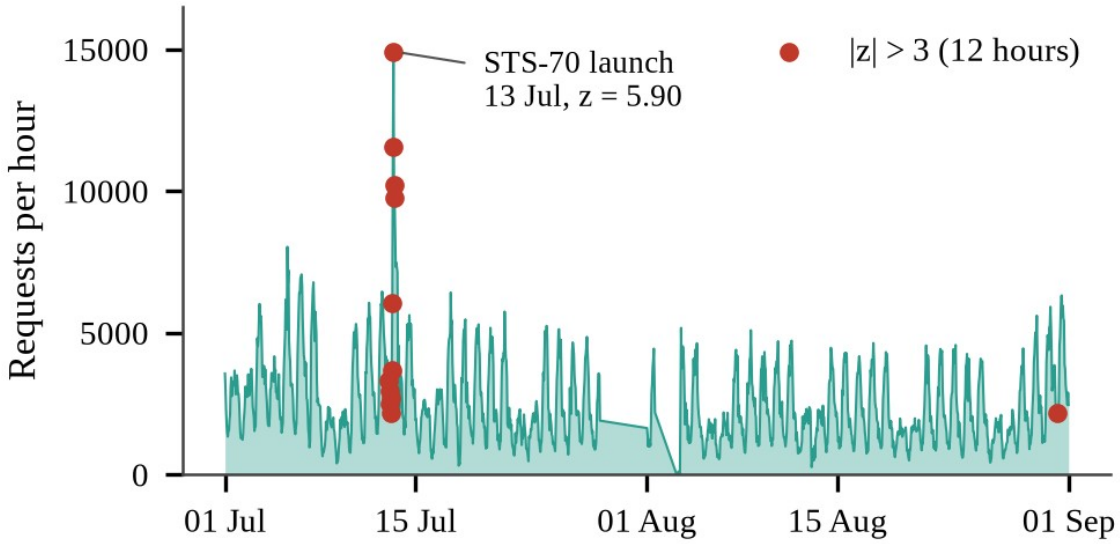


Fig 5.10: Requests per hour with the 12 anomalous hours (|z| > 3)

5.6 Bot Detection Results

The Gaussian Mixture Model was fitted on the 38,104 hosts with at least 20 requests and reached a silhouette coefficient of 0.49. Table 5.4 and Fig 5.11 describe the three components. Two components are human: they load many images and browse at a fast pace. The third component, 1,829 hosts, behaves like software: 2.5% of its requests are HEAD requests (against 0.04% and 0.10% for humans), its median gap between requests is 18 seconds, and only a third of its requests are images. These automated hosts produce 6.36% of all requests.

Table 5.4: The three components of the Gaussian Mixture Model

Component	Hosts	Share of requests	HEAD %	Median gap (s)	Images %
C0 human	32,503	52.3%	0.04	5	63.2
C1 human	3,772	41.4%	0.10	2	70.2
C2 automated	1,829	6.4%	2.50	18	33.5

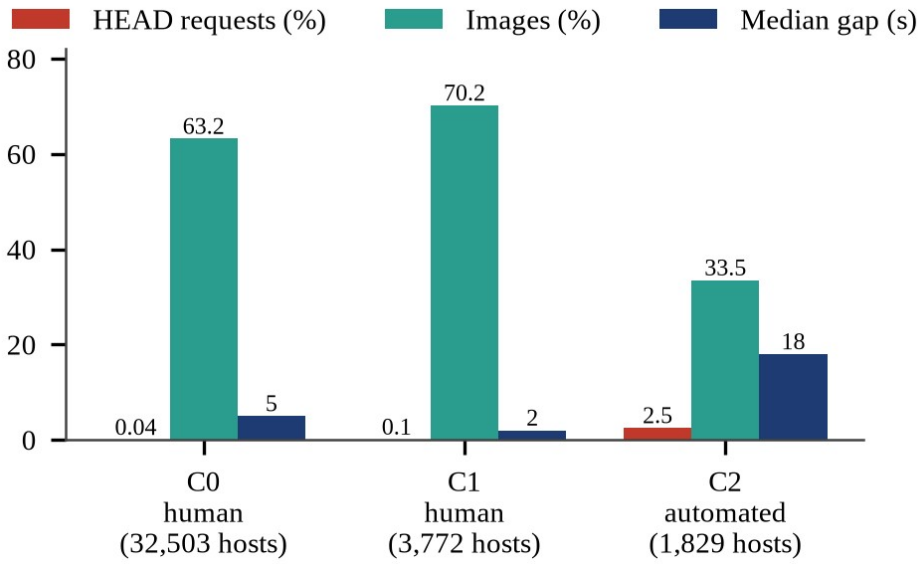


Fig 5.11: Behaviour of the three GMM components

As a held-out check, 27 hosts requested /robots.txt, a strong sign of a crawler. The automated component contains 15 of them — a recall of 55.6% and 11.6 times the rate expected by chance. Because most crawlers never request robots.txt, this recall is a lower bound.

5.7 Attack Detection Results

5.7.1 Comparison with the Existing Classifiers

Table 5.5 and Fig 5.12 compare the eight Spark MLlib models on the same 261,076 test requests (253,411 normal and 7,665 attack). The Factorization Machine reaches an F1 of 0.9669, within 0.002 of the linear SVM and ahead of logistic regression, the decision tree, gradient-boosted trees, Naive Bayes, the multilayer perceptron and the random forest. Every model reaches an

accuracy above 99%, so accuracy does not separate them, while F1 spreads them from 0.816 to 0.969. The random forest has high precision but misses 29% of attacks.

Table 5.5: Eight Spark MLlib classifiers on the same test set

Model	Precision	Recall	F1	FP	FN	Time (s)
Linear SVM	0.9494	0.9885	0.9686	404	88	4.7
Factorization Machine (ours)	0.9498	0.9846	0.9669	399	118	67.4
Logistic Regression	0.9456	0.9855	0.9651	435	111	3.6
Decision Tree	0.9292	0.9586	0.9437	560	317	27.4
Gradient Boosted Trees	0.8190	0.9632	0.8853	1,632	282	131.6
Naive Bayes	0.9146	0.8033	0.8553	575	1,508	0.5
Multilayer Perceptron	0.7787	0.9387	0.8512	2,045	470	43.6
Random Forest	0.9571	0.7109	0.8158	244	2,216	24.5

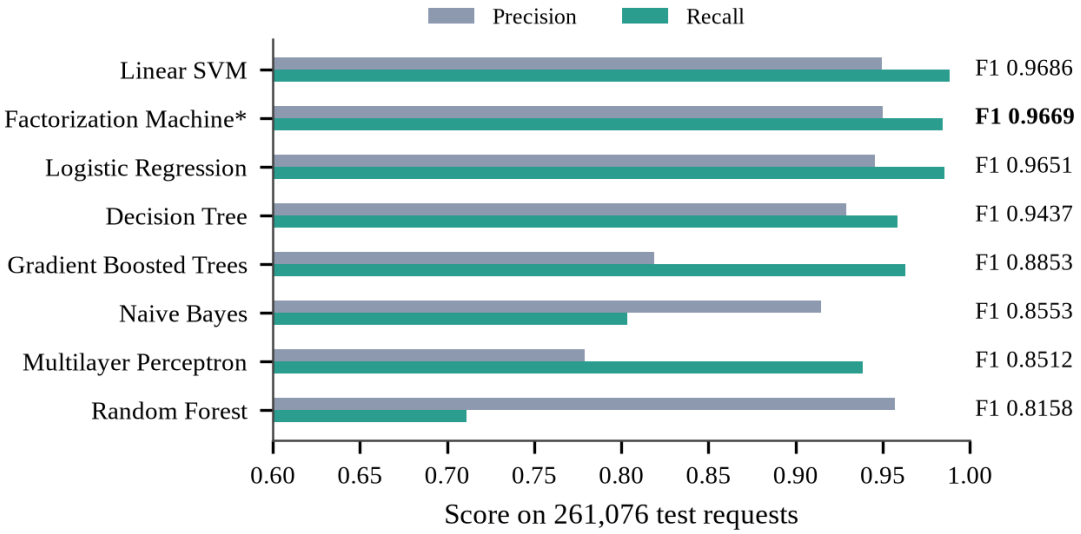


Fig 5.12: Precision, recall and F1 of the eight models

5.7.2 Results of the Proposed Model

Table 5.6 shows the confusion matrix of the Factorization Machine. It catches 7,547 of the 7,665 attacks (recall 98.5%) and misses 118; of the 253,411 normal requests it wrongly flags 399 (0.16%), so 95.0% of its alarms are real attacks. Its accuracy is 99.80%. Reloaded from HDFS, the saved model reproduces the same F1 of 0.9669.

Table 5.6: Confusion matrix of the Factorization Machine (test set)

	Predicted normal	Predicted attack
Actual normal	253,012 (TN, 99.84%)	399 (FP, 0.16%)
Actual attack	118 (FN, 1.54%)	7,547 (TP, 98.46%)

5.7.3 Stability and Transfer to New Data

One split can be lucky, so the experiment was repeated on five random splits (Table 5.7). Inside one dataset the linear SVM is the most accurate and most stable model (F1 0.9697 ± 0.0007), followed by logistic regression (0.9663) and the Factorization Machine (0.9611 ± 0.0062), whose factors start from random values. McNemar's test finds the SVM's advantage over the Factorization Machine significant ($p < 0.05$) on four of the five splits.

A mixed split lets a model learn the style of each dataset, so the models were also trained on one dataset and tested on the other (Table 5.7, Fig 5.13). Trained on Fwaf and tested on HttpParams, the ranking reverses: the Factorization Machine keeps an F1 of 0.968, against 0.897 for logistic regression and 0.846 for the SVM, with about 271 false alarms against 1,757 and 3,777 — 6 to 14 times fewer. Trained only on the small HttpParams set, every model fails on Fwaf's full URLs, with F1 below 0.42.

Table 5.7: Stability over five splits and transfer across datasets (F1)

Model	5 random splits	Fwaf → HttpParams	HttpParams → Fwaf
Linear SVM	0.9697 ± 0.0007	0.846 ± 0.012	0.303 ± 0.010
Logistic Regression	0.9663 ± 0.0009	0.897 ± 0.011	0.411 ± 0.000
Factorization Machine (ours)	0.9611 ± 0.0062	0.968 ± 0.008	0.169 ± 0.045

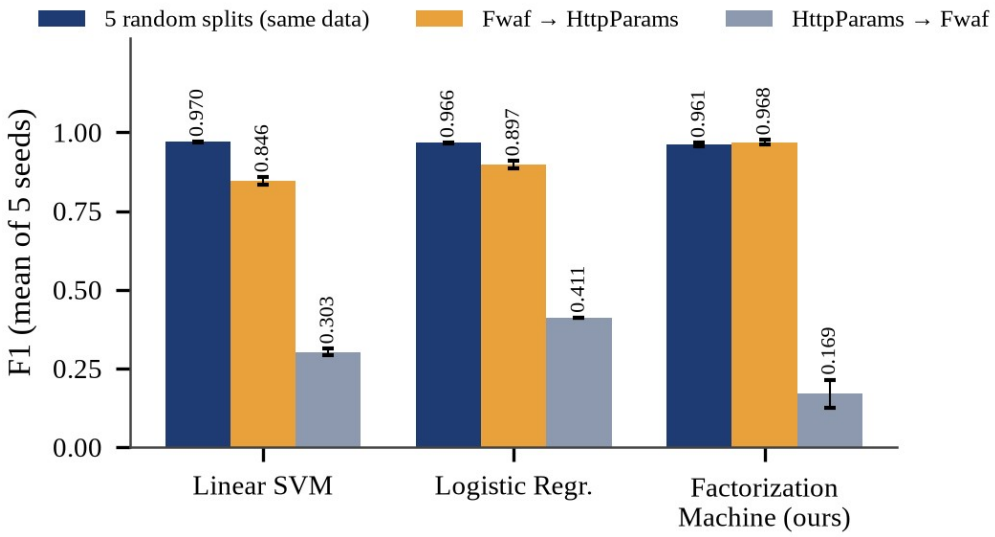


Fig 5.13: F1 within one dataset and across datasets

5.7.4 Case Study on the Unlabelled NASA Logs

The deployed model scored all 31,026 distinct NASA URLs and flagged 120 of them (1,754 requests, 0.05% of all traffic). Every flagged URL was checked by hand (Table 5.8). None contains a real attack signature such as `union select`, `<script`, `../..` or `/etc/passwd`: most are site searches containing words such as *select*, and pages under `/msfc/description/`, whose

path shares character n-grams with `<script`. Without labels this is a qualitative check of the shift between modern attack data and 1995 traffic, not a precision measurement.

Table 5.8: Hand audit of the flagged NASA URLs

What the URL really is	URLs	Requests
Site search (wais.pl)	44	55
Page under /msfc/description/	35	933
Malformed or mistyped request	21	38
Other ordinary page or file	11	708
CGI with parameters	9	20

5.8 Scalability

To measure scaling, the ETL and the eleven reports were run on 25%, 50%, 100% and 200% of the logs with one and two cores (Table 5.9, Fig 5.14). Doubling the data from 3.46 to 6.92 million lines raised the time on two cores from 123 s to 187 s, so throughput rose to 36,968 lines per second as Spark's fixed start-up cost was spread over more data, and the second core helped more as the data grew (speed-up 1.15 at 25% and 1.47 at 200%).

Table 5.9: Run time of the ETL and the eleven reports

Data	Lines	Size (MB)	Cores	Time (s)	Lines / s
25%	865,403	88.1	1	60.9	14,220
25%	865,403	88.1	2	53.1	16,296
50%	1,730,806	177.0	1	92.6	18,685
50%	1,730,806	177.0	2	74.6	23,214
100%	3,461,613	355.8	1	161.0	21,498
100%	3,461,613	355.8	2	123.0	28,140
200%	6,923,225	711.5	1	275.9	25,096
200%	6,923,225	711.5	2	187.3	36,968

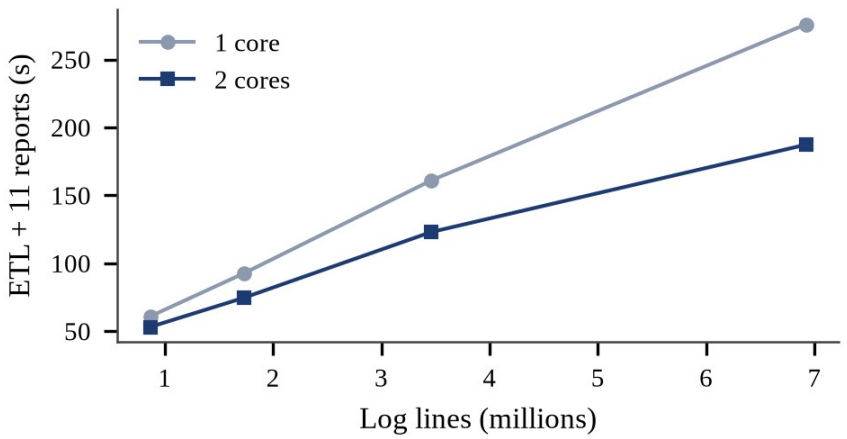


Fig 5.14: Run time against data size with one and two cores

5.9 Visualization Results

All results are presented through Apache Superset dashboards that query HDFS through the Spark Thrift Server. Fig 5.15 shows the Traffic dashboard and Fig 5.16 the Anomalies dashboard, with requests per hour over both months, the twelve unusual hours against their normal level and the anomaly table. Fig 5.17 shows the Bot detection dashboard with the share of hosts and requests per segment, the behaviour of the three components and the list of automated hosts. Fig 5.18 shows the Attack detection dashboard with the F1 of the eight models, precision and recall, the confusion matrix with a model filter and the stability charts.

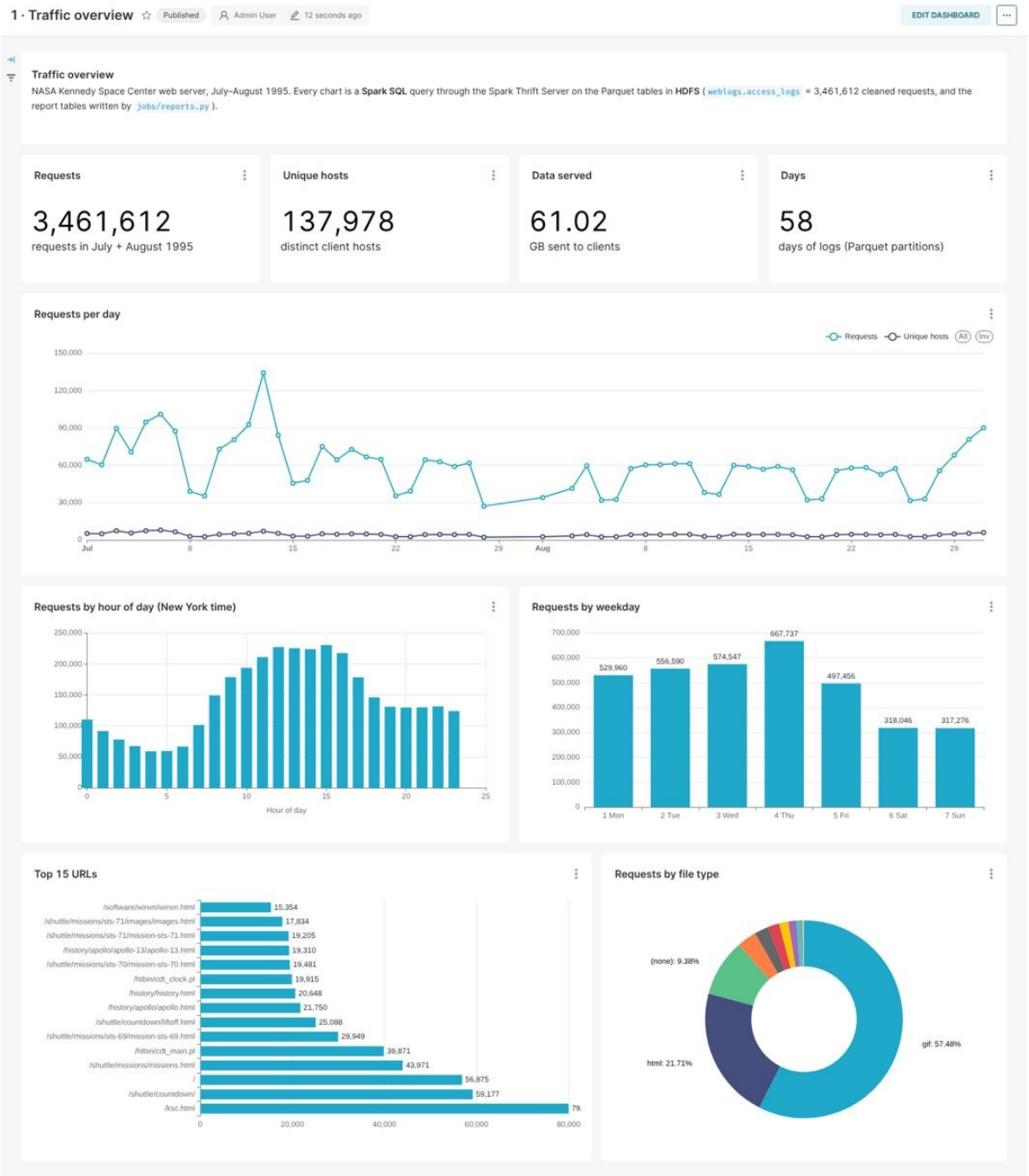


Fig 5.15: Traffic dashboard in Apache Superset

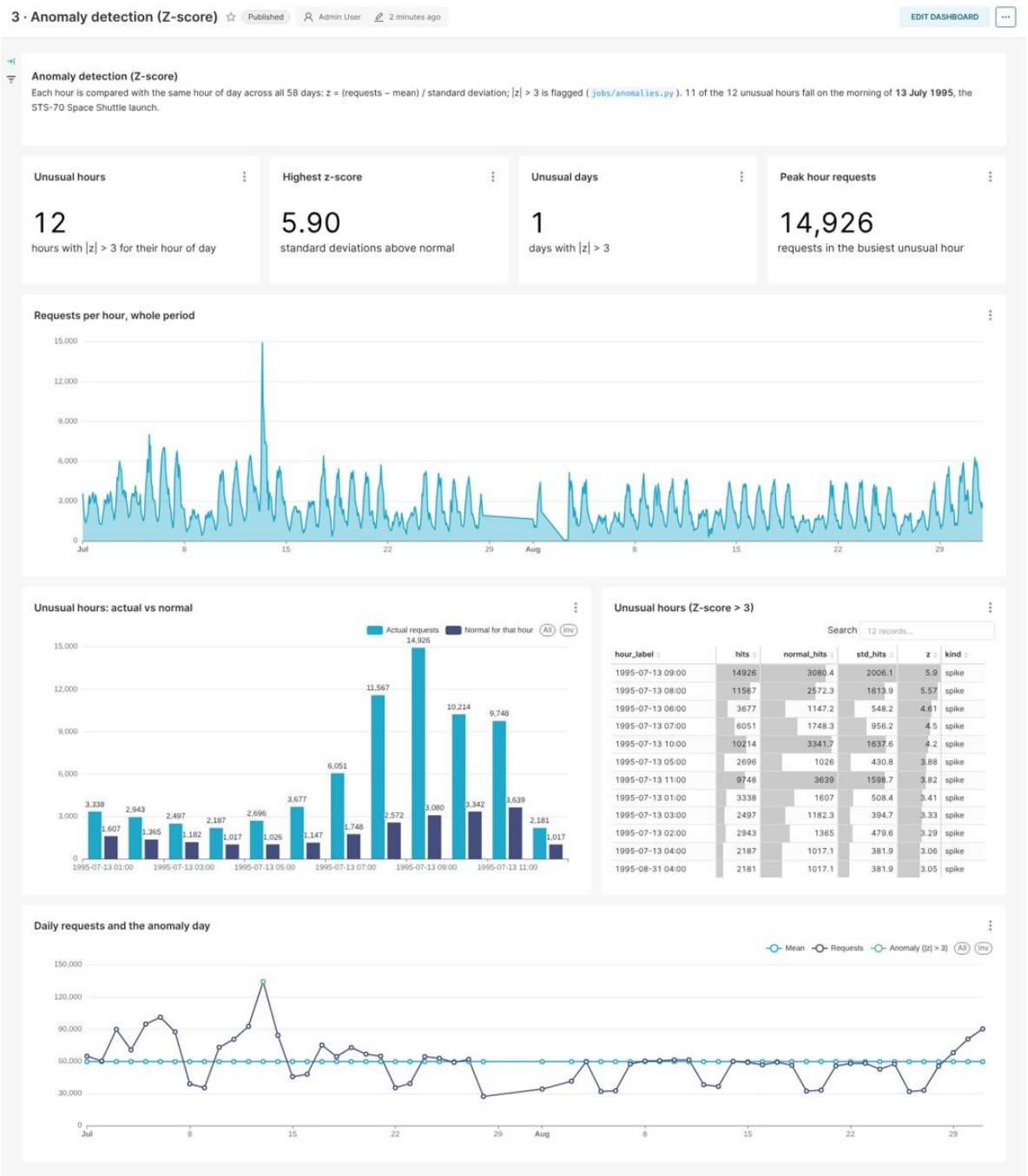


Fig 5.16: Anomalies (Z-score) dashboard in Apache Superset

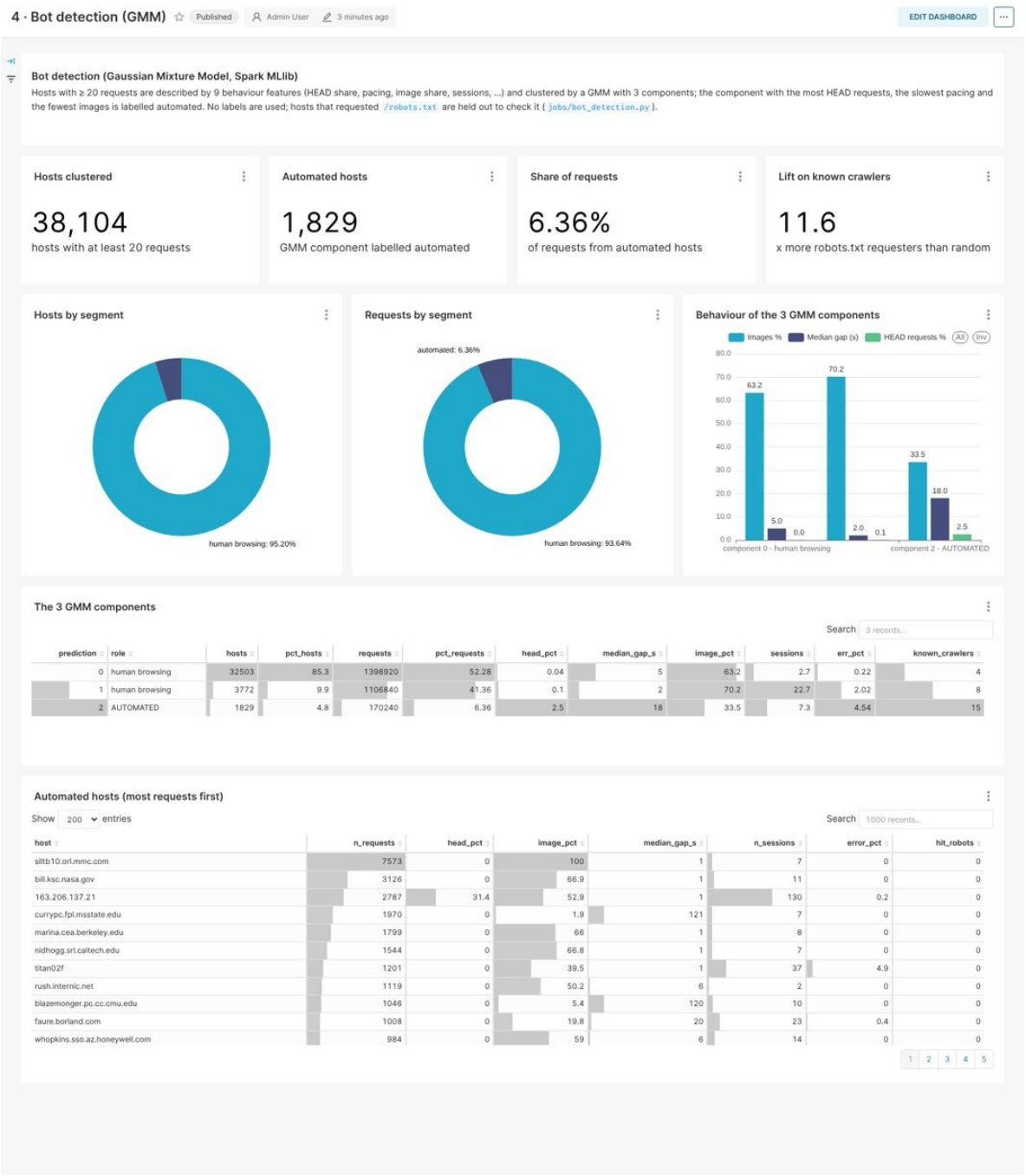


Fig 5.17: Bot detection (GMM) dashboard in Apache Superset

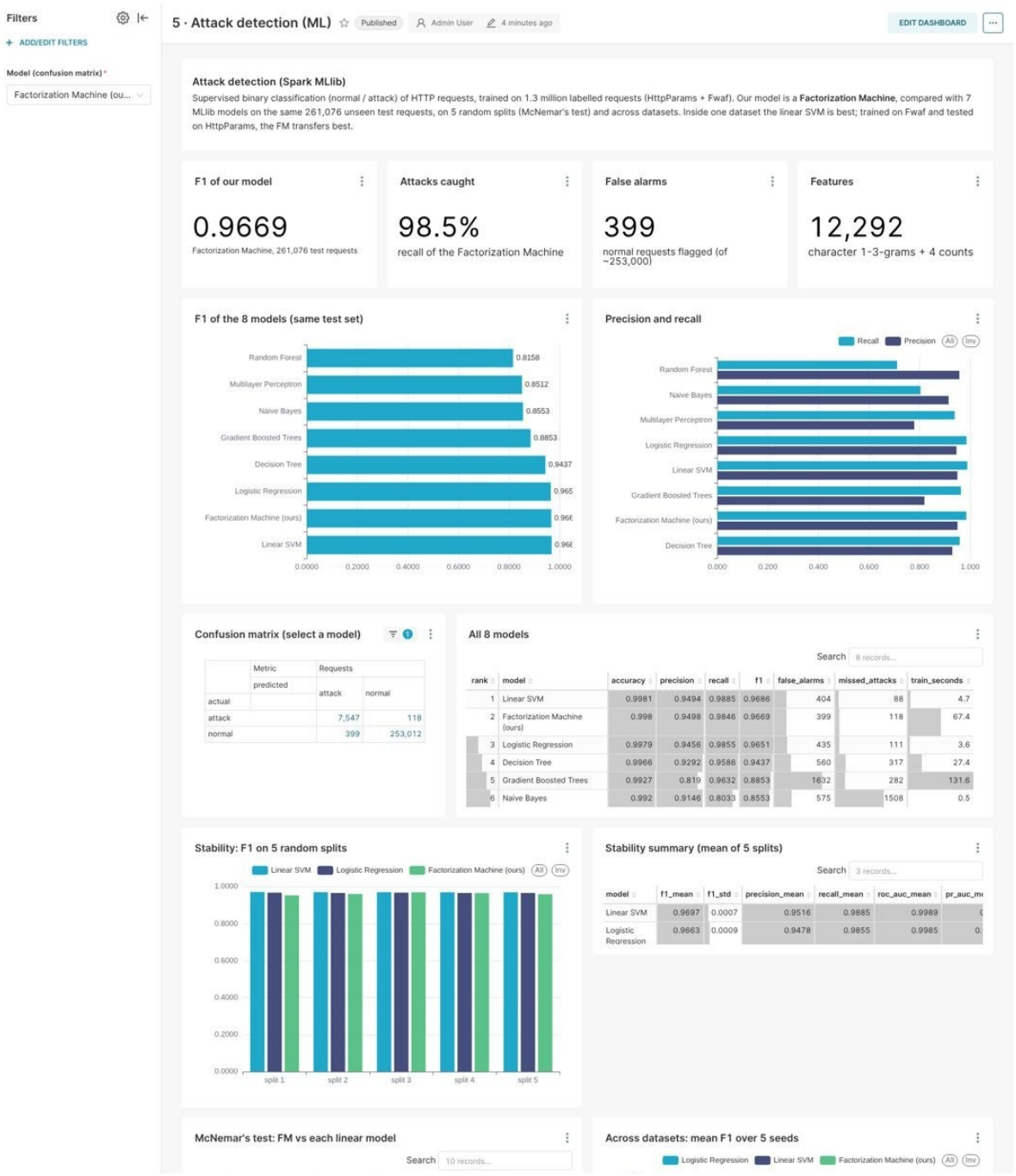


Fig 5.18: Attack detection (ML) dashboard in Apache Superset

Fig 5.19 shows the LogLens home page, with headline numbers computed by Spark on HDFS, quick actions, recent checks and key takeaways above an embedded Superset dashboard. Fig 5.20 shows the *Check a request* page: a request containing a script tag is classified as an attack with 100% probability, while a normal NASA search scores 0.04%.

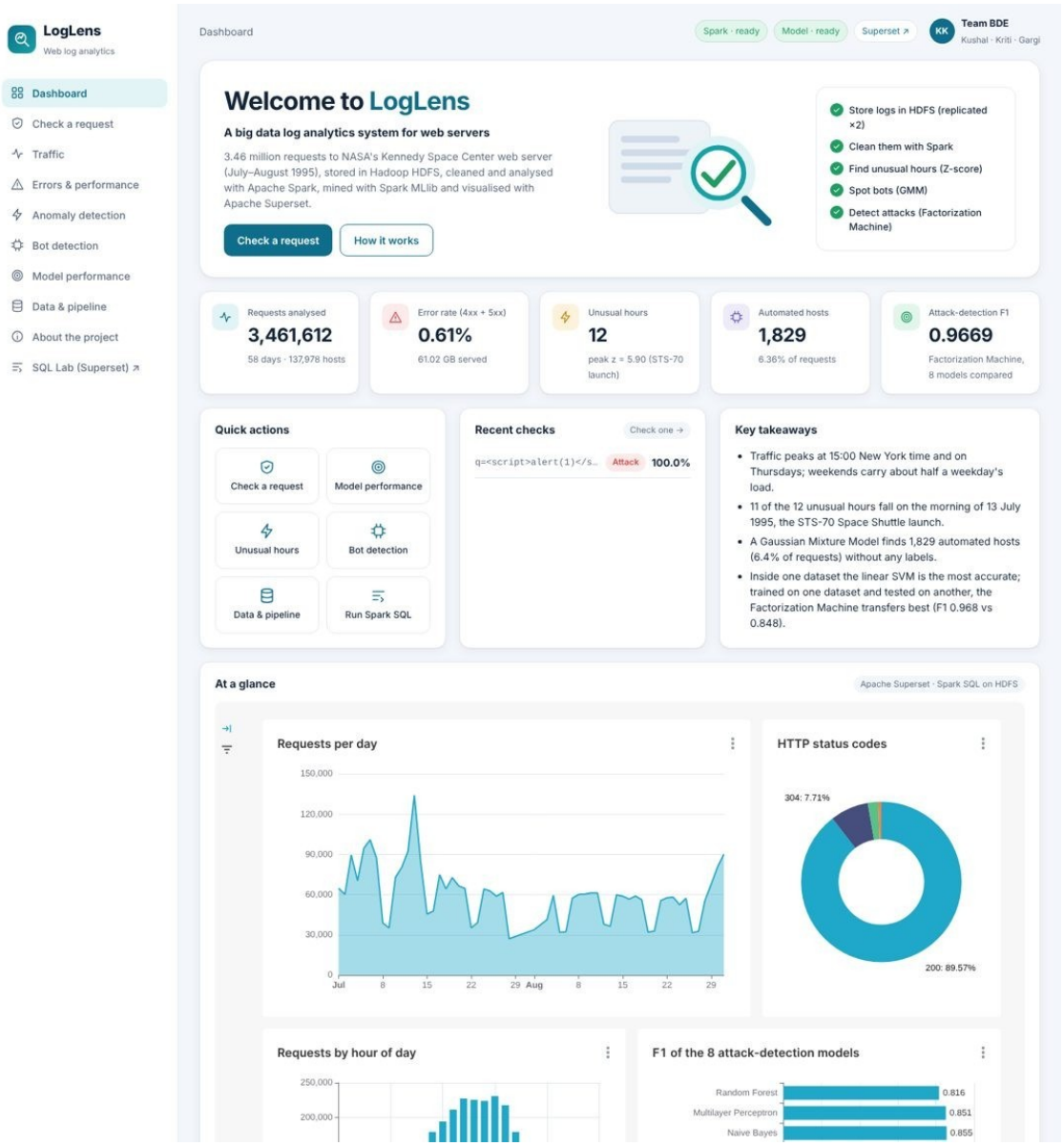


Fig 5.19: LogLens home page with an embedded Superset dashboard

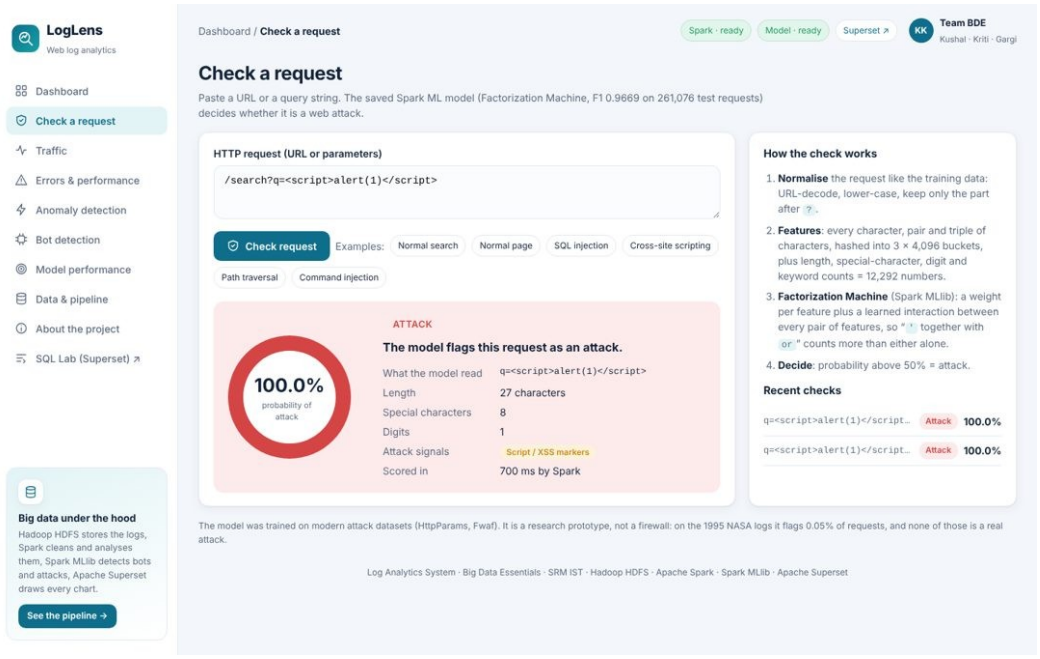


Fig 5.20: LogLens 'Check a request' page running the saved model

5.10 Discussion of Results

The results show that the system meets its objectives. All 3.46 million lines were processed with every loss accounted for, the replicated storage survived the loss of a DataNode, and every chart is a Spark SQL query on an HDFS table. The traffic analysis produced findings that are directly actionable for a site owner: capacity should be planned for weekday afternoons, two broken files cause 18% of all 404 errors, and bandwidth is dominated by images and a few video files. These agree with the workload characteristics reported by Arlitt and Williamson [7] for the same server, now confirmed on the full two months with a distributed engine.

The two unsupervised detectors found real structure without labels: the Z-score recovered the STS-70 launch, and the GMM separated a small automated population confirmed by the robots.txt check. The attack study shows why measurement matters. Before the bias fix the same models scored higher and an earlier model flagged 1,083 NASA URLs, mostly ordinary searches, because it had learned that a query string signals an attack. After the fix, the within-dataset ranking (SVM ahead of the Factorization Machine, significant on most splits) and the cross-dataset ranking (Factorization Machine far ahead) disagree. The Factorization Machine is deployed because a detector always meets traffic it was not trained on; its factorised pairwise terms capture n-grams that occur together in attacks rather than dataset-specific single n-grams.

The main limitations are that the NASA logs have no labels, so detection on them is a qualitative case study; that both labelled datasets are small next to real traffic and come from neither the same server nor the same era; that the cluster is distributed in software but runs on one laptop, so scaling was measured only up to 712 MB; and that the Factorization Machine's run-to-run variance (F1 standard deviation 0.006) is larger than that of the linear models.

5.11 Summary of Findings

- 3,461,613 log lines were processed into 3,461,612 typed rows plus 1 reject, from 137,978 hosts to 31,030 URLs, totalling 61.02 GB; the curated data is 8.6 times smaller than the raw text.
- Traffic peaks at 15:00 (230,665 requests) and on Thursdays; 89.6% of requests succeeded, and two files cause 18% of the 20,899 404 errors.
- The contextual Z-score flagged 12 unusual hours, 11 of them on 13 July 1995; the peak ($z = 5.90$) is the STS-70 launch hour.
- The GMM found 1,829 automated hosts producing 6.36% of requests, with an 11.6× lift on known crawlers, without labels.
- The Factorization Machine detects attacks with F1 0.9669 (recall 98.5%, 399 false alarms); the linear SVM is slightly better within one dataset, but the FM transfers best to new data (F1 0.968 against 0.846).
- All results are served by 27 Spark Thrift Server tables, 6 Superset dashboards with 56 charts and the LogLens application; 35 automated tests pass.

CHAPTER 6
CONCLUSION AND FUTURE WORK

This report presented the complete Log Analytics System for a Web Server. A distributed system was designed and implemented on a real big-data stack: a ten-container Docker cluster with HDFS for replicated storage, Apache Spark and Spark SQL for processing, Spark MLlib for machine learning, and Apache Superset on the Spark Thrift Server for visualisation. Two months of NASA web server logs — 3,461,613 lines — were ingested into an immutable raw zone, cleaned into typed, date-partitioned Parquet with explicit handling of malformed lines, and analysed with eleven Spark SQL reports. A contextual Z-score recovered the STS-70 launch from the traffic alone, and a Gaussian Mixture Model separated 1,829 automated hosts without labels.

For attack detection, a Factorization Machine was trained on 1.31 million labelled requests after a measured dataset shortcut was removed. It detects attacks with an F1 of 0.967, close to the best of seven comparison models within one dataset, and it transfers clearly best to a dataset it was never trained on. Every result is stored in HDFS, served by the Spark Thrift Server and drawn by six Superset dashboards, which the LogLens application embeds together with a live attack checker that runs the saved model. The system is reproducible with a few commands, covered by 35 automated tests, and was benchmarked on up to twice the original data.

The work was divided by stage of the architecture: Kriti Singh built the Docker cluster, the HDFS storage and the GMM bot detection; Kushal Mohan built the Spark ETL, the preprocessing of the labelled data and the Factorization Machine; and Gargi Singh built the Spark SQL reports, the Z-score anomaly detection, the test on new data and the visualisation layer with the Thrift Server, Superset and LogLens. The comparison study, the tests and the documentation were done together.

Future work that would extend the system is:

- **More realistic labelled data:** add labelled traffic closer to real servers, such as the CSIC 2010 HTTP dataset [33], and hand-label a random sample of flagged URLs to estimate precision on real traffic.
- **A more stable detector:** average several Factorization Machine runs to reduce their variance, and tune the decision threshold on the validation split.
- **Live scoring:** use Spark Structured Streaming to score new traffic in micro-batches instead of a single batch, with new logs arriving through Kafka or Flume.
- **Scale:** run the same pipeline on a multi-node cluster in the cloud with 5 and 10 times the data, and on a modern server's logs with user-agent and response-time fields.
- **Security:** add authentication to the cluster services and Superset roles for different users, since the current cluster is meant for local use.

Through this project the team learned how large, raw data can be stored safely, processed at scale, mined with distributed machine learning and turned into answers that people can use — and that measuring the data and testing on new data are as important as the model itself.

REFERENCES

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI), San Francisco, CA, USA, 2004, pp. 137–150.

[2] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The Google file system,” in Proc. 19th ACM Symp. Operating Systems Principles (SOSP), 2003, pp. 29–43.

[3] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop distributed file system,” in Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST), 2010, pp. 1–10.

[4] M. Zaharia et al., “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2012, pp. 15–28.

[5] M. Armbrust et al., “Spark SQL: Relational data processing in Spark,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2015, pp. 1383–1394.

[6] S. Melnik et al., “Dremel: Interactive analysis of web-scale datasets,” Proc. VLDB Endowment, vol. 3, no. 1–2, pp. 330–339, 2010.

[7] M. F. Arlitt and C. L. Williamson, “Internet web servers: Workload characterization and performance implications,” IEEE/ACM Trans. Networking, vol. 5, no. 5, pp. 631–645, 1997.

[8] R. Cooley, B. Mobasher, and J. Srivastava, “Data preparation for mining World Wide Web browsing patterns,” Knowledge and Information Systems, vol. 1, no. 1, pp. 5–32, 1999.

[9] J. Srivastava, R. Cooley, M. Deshpande, and P.-N. Tan, “Web usage mining: Discovery and applications of usage patterns from web data,” SIGKDD Explorations, vol. 1, no. 2, pp. 12–23, 2000.

[10] M. Greenwald and S. Khanna, “Space-efficient online computation of quantile summaries,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2001, pp. 58–66.

[11] A. Oliner, A. Ganapathi, and W. Xu, “Advances and challenges in log analysis,” Communications of the ACM, vol. 55, no. 2, pp. 55–61, 2012.

[12] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, “Detecting large-scale system problems by mining console logs,” in Proc. 22nd ACM Symp. Operating Systems Principles (SOSP), 2009, pp. 117–132.

[13] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in Proc. ACM Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298.

[14] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, 2009.

[15] D. A. Reynolds, “Gaussian mixture models,” in Encyclopedia of Biometrics, S. Z. Li and A. Jain, Eds. Boston, MA, USA: Springer, 2009, pp. 659–663.

[16] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” J. Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.

[17] S. Rendle, “Factorization machines,” in Proc. IEEE Int. Conf. Data Mining (ICDM), 2010, pp. 995–1000.

[18] X. Meng et al., “MLlib: Machine learning in Apache Spark,” J. Machine Learning Research, vol. 17, no. 34, pp. 1–7, 2016.

[19] J. Saxe and K. Berlin, “eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys,” arXiv:1702.08568, 2017.

[20] R. Geirhos et al., “Shortcut learning in deep neural networks,” Nature Machine Intelligence, vol. 2, no. 11, pp. 665–673, 2020.

[21] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[22] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[23] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[24] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[25] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[26] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” Neural Computation, vol. 10, no. 7, pp. 1895–1923, 1998.

[27] Apache Software Foundation, “Apache Superset documentation.” [Online]. Available: https://superset.apache.org/docs/

[28] Apache Software Foundation, “Spark SQL: Distributed SQL engine (Thrift JDBC/ODBC server),” Spark 3.5.3 documentation. [Online]. Available: https://spark.apache.org/docs/3.5.3/sql-distributed-sql-engine.html

[29] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” Linux Journal, vol. 2014, no. 239, Art. no. 2, 2014.

[30] Internet Traffic Archive, “NASA-HTTP: Two months of HTTP logs from the KSC-NASA WWW server.” [Online]. Available: https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html

[31] Morzeux, “HttpParamsDataset,” GitHub repository. [Online]. Available: https://github.com/Morzeux/HttpParamsDataset

[32] F. Ahmad, “Fwaf: Machine learning driven web application firewall,” GitHub repository, 2017. [Online]. Available: https://github.com/faizann24/Fwaf-Machine-Learning-driven-Web-Application-Firewall

[33] C. Torrano Giménez, A. Pérez Villegas, and G. Álvarez Marañón, “HTTP data set CSIC 2010,” Information Security Institute, CSIC, Madrid, Spain, 2010.